

# **Zaawansowane Technologie Baz Danych**

Porównanie wydajności systemów zarządzania bazami danych:  
PostgreSQL, MySQL, MongoDB, Redis

Sprawozdanie projektowe

Jakub Balicki

Politechnika Krakowska

Kierunek: Informatyka

Maj 2026

# Spis treści

|          |                                                                                      |           |
|----------|--------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Cel i zakres pracy</b>                                                            | <b>4</b>  |
| 1.1      | Hipoteza badawcza . . . . .                                                          | 4         |
| <b>2</b> | <b>Opis wybranych systemów zarządzania bazami danych</b>                             | <b>4</b>  |
| 2.1      | PostgreSQL 16 . . . . .                                                              | 4         |
| 2.2      | MySQL 8.0 . . . . .                                                                  | 4         |
| 2.3      | MongoDB 7 . . . . .                                                                  | 5         |
| 2.4      | Redis 7 . . . . .                                                                    | 5         |
| <b>3</b> | <b>Opis zbioru danych</b>                                                            | <b>5</b>  |
| 3.1      | Modele danych w systemach nierelacyjnych . . . . .                                   | 7         |
| 3.2      | Skale danych testowych . . . . .                                                     | 7         |
| <b>4</b> | <b>Opis aplikacji testowej</b>                                                       | <b>7</b>  |
| 4.1      | Wymagania . . . . .                                                                  | 7         |
| 4.2      | Wykorzystane technologie i narzędzia . . . . .                                       | 7         |
| 4.3      | Struktura projektu i opis działania . . . . .                                        | 8         |
| <b>5</b> | <b>Opis scenariuszy testowych</b>                                                    | <b>8</b>  |
| <b>6</b> | <b>Metodyka pomiaru</b>                                                              | <b>10</b> |
| 6.1      | Schemat pomiaru . . . . .                                                            | 10        |
| 6.2      | Środowisko testowe . . . . .                                                         | 10        |
| 6.3      | Ograniczenia i zastrzeżenia interpretacyjne . . . . .                                | 11        |
| <b>7</b> | <b>Wyniki testów wydajnościowych</b>                                                 | <b>11</b> |
| 7.1      | Operacja wstawiania danych . . . . .                                                 | 11        |
| 7.2      | Operacja odczytu danych . . . . .                                                    | 15        |
| 7.3      | Operacja aktualizacji danych . . . . .                                               | 20        |
| 7.4      | Operacja usuwania danych . . . . .                                                   | 24        |
| <b>8</b> | <b>Analiza indeksów</b>                                                              | <b>28</b> |
| 8.1      | Zastosowane indeksy . . . . .                                                        | 28        |
| 8.2      | Analiza planów wykonania zapytań . . . . .                                           | 28        |
| 8.2.1    | PostgreSQL – pobranie diagnoz wizyty (10M, złączenie po kluczu obcym) . . . . .      | 28        |
| 8.2.2    | PostgreSQL – wyliczenie sumy kosztów usług (500k, kontrolna analiza planu) . . . . . | 28        |
| 8.2.3    | PostgreSQL – zmiana ceny wykonanej usługi (aktualizacja po kluczu obcym) . . . . .   | 29        |
| 8.2.4    | MySQL 8.0 – pobranie diagnoz wizyty . . . . .                                        | 29        |
| 8.2.5    | MongoDB – pobranie wizyt pacjenta z lekarzem (kolekcja wizyt 2M dok.) . . . . .      | 30        |
| 8.2.6    | Wnioski porównawcze . . . . .                                                        | 30        |

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| <b>9 Weryfikacja hipotezy badawczej</b>                                | <b>30</b> |
| 9.1 Przyspieszenie odczytu ( $S_{odczyt}$ ) – trend ze skalą . . . . . | 31        |
| 9.2 Spowolnienie zapisu ( $K_{zapis}$ ) – trend ze skalą . . . . .     | 32        |
| 9.3 Wnioski dotyczące hipotezy . . . . .                               | 32        |
| <b>10 Podsumowanie i wnioski</b>                                       | <b>33</b> |

# 1 Cel i zakres pracy

Celem projektu jest porównanie wydajności czterech systemów zarządzania bazami danych (SZBD) przy wykonywaniu operacji CRUD: tworzenia, odczytu, aktualizacji i usuwania danych na dużych zbiorach. Analizie poddano:

- **2 systemy relacyjne:** PostgreSQL 16, MySQL 8.0
- **2 systemy nierelacyjne:** MongoDB 7 (dokumentowa), Redis 7 (klucz-wartość)

Zakres pracy obejmuje:

1. Zaprojektowanie schematu danych systemu szpitalnego (13 tabel relacyjnych, model dokumentowy MongoDB, struktury klucz-wartość Redis).
2. Implementację aplikacji testowej automatyzującej generowanie danych, wstawianie, pomiary i analizę.
3. Przeprowadzenie 24 scenariuszy testowych CRUD (6 na każdą operację) w trzech skalach: 500 000, 1 000 000 oraz 10 000 000 sumarycznych rekordów.
4. Porównanie wydajności przed i po zastosowaniu indeksów.
5. Analizę planów wykonania zapytań.
6. Weryfikację hipotezy badawczej o wpływie indeksów na wydajność.

## 1.1 Hipoteza badawcza

**H1: Indeksy a wydajność** – Zastosowanie indeksów znacząco poprawia wydajność odczytu danych kosztem spadku wydajności wstawiania i aktualizacji danych.

# 2 Opis wybranych systemów zarządzania bazami danych

## 2.1 PostgreSQL 16

PostgreSQL jest zaawansowanym, obiektowo-relacyjnym systemem baz danych o otwartym kodzie źródłowym. Oferuje pełną zgodność z ACID, zaawansowane mechanizmy indeksowania (B-tree, Hash, GIN, GiST, BRIN), obsługę JSON/JSONB, widoki zmaterializowane, partycjonowanie tabel oraz replikację logiczną i strumieniową.

**Zalety:** Zaawansowane typy danych, rozbudowany optymalizator zapytań, pełna zgodność ze standardem SQL, doskonała obsługa transakcji, MVCC (Multi-Version Concurrency Control), wsparcie dla procedur składowanych w wielu językach (PL/pgSQL, PL/Python).

**Wady:** Wyższe zużycie pamięci w porównaniu z MySQL, wolniejsze wstawianie danych masowych bez odpowiedniej konfiguracji, bardziej złożona konfiguracja.

**Zastosowania biznesowe:** Systemy finansowe, systemy ERP, analityka danych, systemy GIS (PostGIS), platformy e-commerce.

## 2.2 MySQL 8.0

MySQL jest najpopularniejszym relacyjnym systemem baz danych o otwartym kodzie. Wykorzystuje silnik InnoDB z obsługą transakcji ACID, mechanizmem MVCC, kluczami

obcymi i indeksami B-tree. MySQL 8.0 wprowadził okna analityczne, CTE (Common Table Expressions) i ulepszony optymalizator.

**Zalety:** Wysoka wydajność operacji odczytu, łatwość konfiguracji, szeroka kompatybilność, duża społeczność, niskie wymagania sprzętowe.

**Wady:** Ograniczone wsparcie dla zaawansowanych typów danych, brak niektórych funkcji SQL standardu (np. pełna obsługa sekwencji), historycznie słabsza optymalizacja podzapytań.

**Zastosowania biznesowe:** Aplikacje webowe (WordPress, Joomla), systemy CMS, platformy SaaS, systemy logistyczne.

## 2.3 MongoDB 7

MongoDB jest dokumentową bazą danych NoSQL, przechowującą dane w formacie BSON (Binary JSON). Oferuje elastyczny schemat, replikację zestawów replik (Replica Sets), sharding dla skalowalności horyzontalnej oraz zaawansowane indeksowanie (pojedyncze, złożone, wielokluczowe, tekstowe, geoprzestrzenne).

**Zalety:** Elastyczny schemat (schema-less), naturalne modelowanie danych hierarchicznych, szybkie operacje na pojedynczych dokumentach, łatwa skalowalność horyzontalna, wbudowana agregacja.

**Wady:** Brak transakcji wielodokumentowych (do wersji 4.0), większe zużycie pamięci dyskowej, brak klasycznych złączeń relacyjnych (wymaga denormalizacji lub operatora \$lookup), ewentualna spójność w konfiguracjach rozproszonych.

**Zastosowania biznesowe:** Systemy zarządzania treścią, aplikacje mobilne, IoT, katalogi produktów, systemy analityki czasu rzeczywistego.

## 2.4 Redis 7

Redis (Remote Dictionary Server) jest bazą danych typu klucz-wartość działającą w pamięci RAM. Obsługuje struktury danych: stringi, hashe, listy, zbiory, zbiory uporządkowane, strumienie. Oferuje persystencję (RDB, AOF), replikację master-slave i klaster Redis.

**Zalety:** Ekstremalnie niska latencja ( $< 1$  ms), bogate struktury danych, atomowość operacji, Pub/Sub, obsługa TTL (Time-To-Live), skrypty Lua.

**Wady:** Ograniczona pojemność (limitowana pamięcią RAM), brak złożonych zapytań (brak SQL), brak relacji między danymi, utrata danych w razie awarii (zależy od konfiguracji persystencji).

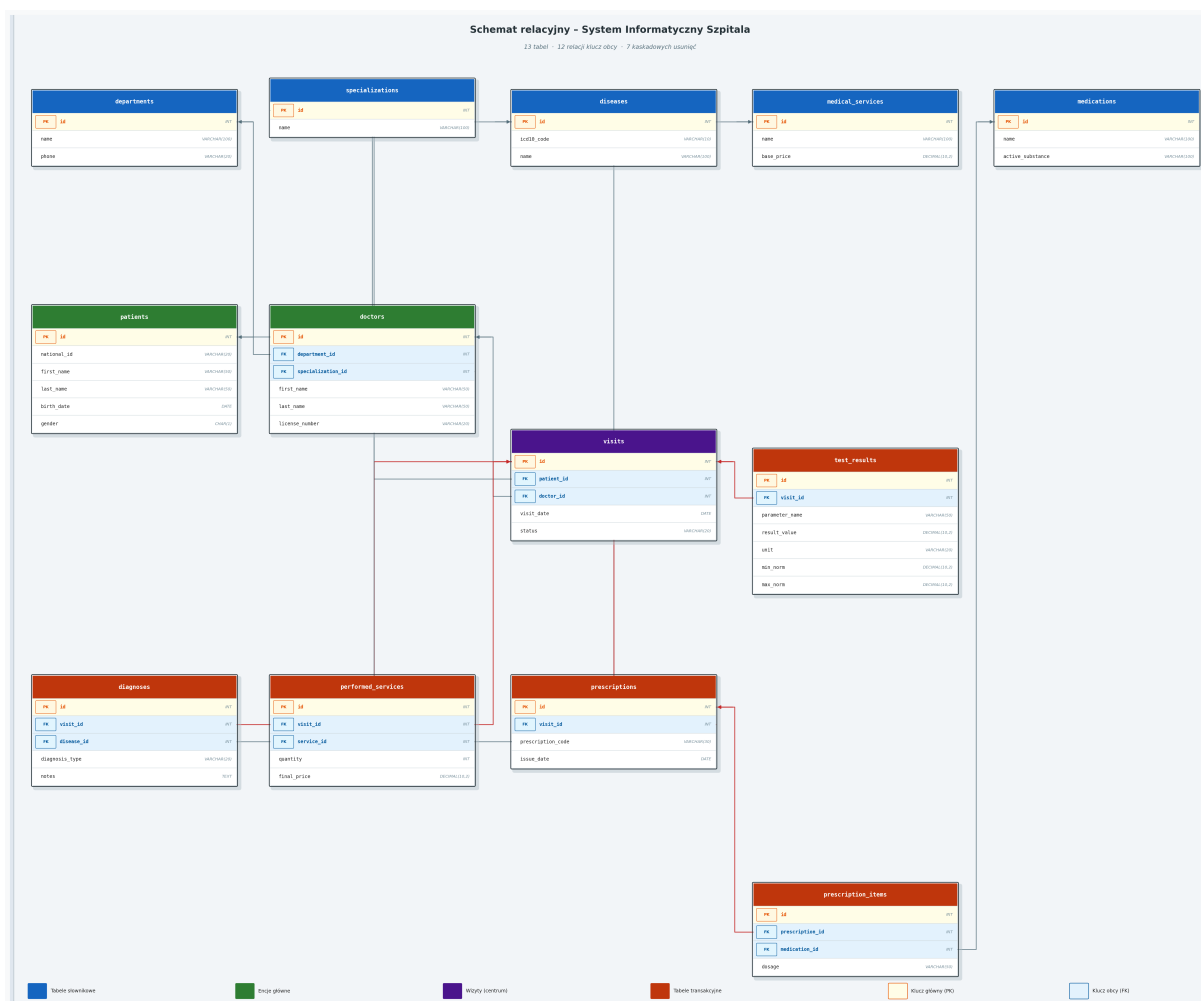
**Zastosowania biznesowe:** Cache'owanie, sesje użytkowników, kolejki wiadomości, tablice wyników w grach, systemy czasu rzeczywistego, liczniki i metryki.

## 3 Opis zbioru danych

Dane testowe modelują system informatyczny szpitala. Schemat relacyjny składa się z 13 tabel:

Tabela 1: Schemat relacyjny – tabele systemu szpitalnego

| Tabela             | Typ          | Opis / klucze obce                                   |
|--------------------|--------------|------------------------------------------------------|
| departments        | Słownikowa   | Oddziały szpitalne (id, name, phone)                 |
| specializations    | Słownikowa   | Specjalizacje lekarskie (id, name)                   |
| diseases           | Słownikowa   | Choroby z kodem ICD-10 (id, icd10_code, name)        |
| medical_services   | Słownikowa   | Usługi medyczne z ceną bazową                        |
| medications        | Słownikowa   | Leki (id, name, active_substance)                    |
| patients           | Główna       | Pacjenci (id, PESEL, imię, nazwisko, data ur., płeć) |
| doctors            | Główna       | Lekarze → departments, specializations               |
| visits             | Transakcyjna | Wizyty → patients, doctors                           |
| performed_services | Transakcyjna | Wykonane usługi → visits, medical_services           |
| diagnoses          | Transakcyjna | Diagnozy → visits, diseases                          |
| prescriptions      | Transakcyjna | Recepty → visits                                     |
| prescription_items | Transakcyjna | Pozycje recept → prescriptions, medications          |
| test_results       | Transakcyjna | Wyniki badań → visits                                |



Rysunek 1: Diagram ERD – schemat relacyjny systemu informatycznego szpitala

### 3.1 Modele danych w systemach nierelacyjnych

**MongoDB** – model dokumentowy z zagnieżdżeniami: Pacjent jest dokumentem głównym, zawierającym tablicę wizyt, a każda wizyta posiada zagnieżdżone tablice: wykonane usługi, diagnozy, recepty (z pozycjami) oraz wyniki badań. Taki model eliminuje konieczność złączeń relacyjnych, ale komplikuje aktualizację zagnieżdżonych struktur.

**Redis** – model klucz-wartość z wieloma typami struktur:

- `patient:{id}` – HASH z danymi pacjenta,
- `patient:visits:{pid}` – SET identyfikatorów wizyt,
- `visit:status:{id}`, `visit:doctor:{vid}` – STRING,
- `session:doctor:{id}` – HASH z danymi lekarza,
- `visit:diag:{vid}` – LIST diagnoz,
- `prescription:{id}`, `service:total:{vid}`, `department:{id}` – HASH.

### 3.2 Skale danych testowych

Tabela 2: Liczba rekordów w zależności od skali

| Tabela             | 500 000  | 1 000 000 | 10 000 000 |
|--------------------|----------|-----------|------------|
| departments        | 10       | 15        | 30         |
| specializations    | 15       | 20        | 40         |
| diseases           | 100      | 150       | 500        |
| medical_services   | 50       | 75        | 200        |
| medications        | 80       | 120       | 400        |
| patients           | 8 000    | 16 000    | 160 000    |
| doctors            | 40       | 80        | 800        |
| visits             | 100 000  | 200 000   | 2 000 000  |
| performed_services | ~100 000 | ~200 000  | ~2 000 000 |
| diagnoses          | ~100 000 | ~200 000  | ~2 000 000 |
| prescriptions      | ~40 000  | ~80 000   | ~800 000   |
| prescription_items | ~80 000  | ~160 000  | ~1 600 000 |
| test_results       | ~60 000  | ~120 000  | ~1 200 000 |

## 4 Opis aplikacji testowej

### 4.1 Wymagania

- Automatyczne generowanie syntetycznych danych medycznych (własny generator).
- Masowe wstawianie danych do 4 systemów bazodanowych.
- Wykonanie 24 scenariuszy CRUD w trybie bez indeksów i z indeksami.
- Pomiar czasu operacji (średnia z 3 prób) z dokładnością do mikrosekundy.
- Automatyczne generowanie raportów planów wykonania i wykresów porównawczych.

### 4.2 Wykorzystane technologie i narzędzia

- **Python 3.13** – język implementacji aplikacji.

- **Docker + Docker Compose** – konteneryzacja wszystkich 4 baz danych.
- **psycopg2** – sterownik PostgreSQL.
- **mysql-connector-python** – sterownik MySQL.
- **pymongo** – sterownik MongoDB.
- **redis-py** – sterownik Redis.
- **pandas + matplotlib** – analiza wyników i wizualizacja.
- **L<sup>A</sup>T<sub>E</sub>X** – skład sprawozdania.

### 4.3 Struktura projektu i opis działania

Aplikacja działa w pełni automatycznie. Polecenie `python run_all.py` kolejno:

1. Sprawdza połączenia z bazami danych (ping).
2. Dla każdej skali (500k, 1M, 10M): generuje dane, wstawia je do baz, uruchamia pomiary bez indeksów i z indeksami oraz generuje plany wykonania zapytań.
3. Łączy wyniki CSV i generuje wykresy porównawcze dla poszczególnych scenariuszy.

## 5 Opis scenariuszy testowych

Zdefiniowano **24 scenariusze testowe** – po 6 dla każdej operacji CRUD. Każdy scenariusz wykonywany jest 3 razy, a raportowana jest średnia arytmetyczna czasu wykonania. Scenariusze zaprojektowano tak, aby były logicznie porównywalne między technologiami.

Tabela 3: Scenariusze testowe – operacje wstawiania danych

| Nr | Nazwa                          | Opis                                                                        |
|----|--------------------------------|-----------------------------------------------------------------------------|
| C1 | Wstawienie pacjenta            | Dodanie nowego pacjenta z podstawowymi danymi osobowymi.                    |
| C2 | Wstawienie wizyty              | Dodanie wizyty powiązanej z pacjentem i lekarzem.                           |
| C3 | Wstawienie diagnozy            | Dodanie diagnozy przypisanej do konkretnej wizyty.                          |
| C4 | Wstawienie recepty z pozycjami | Dodanie recepty oraz powiązanych pozycji leków w jednej operacji logicznej. |
| C5 | Wstawienie wykonanej usługi    | Dodanie usługi medycznej wykonanej podczas wizyty.                          |
| C6 | Wstawienie wyniku badania      | Dodanie wyniku badania laboratoryjnego przypisanego do wizyty.              |



Tabela 4: Scenariusze testowe – operacje odczytu danych

| Nr | Nazwa                                | Opis                                                                                     |
|----|--------------------------------------|------------------------------------------------------------------------------------------|
| R1 | Pobranie pacjenta po identyfikatorze | Odczyt danych jednego pacjenta wskazanego kluczem głównym lub identyfikatorem dokumentu. |
| R2 | Pobranie wizyt pacjenta z lekarzem   | Odczyt wizyt wybranego pacjenta wraz z podstawowymi danymi lekarza.                      |
| R3 | Pobranie diagnoz wizyty              | Odczyt diagnoz przypisanych do jednej wizyty wraz z informacją słownikową o chorobie.    |
| R4 | Pobranie pełnej historii pacjenta    | Odczyt pacjenta, wizyt, diagnoz, recept, usług i wyników badań.                          |
| R5 | Wyliczenie sumy kosztów usług        | Obliczenie łącznego kosztu usług medycznych wykonanych dla pacjenta.                     |
| R6 | Pobranie recept wraz z lekami        | Odczyt recept przypisanych do wizyty oraz powiązanych pozycji leków.                     |

Tabela 5: Scenariusze testowe – operacje aktualizacji i usuwania danych

| Nr | Nazwa                          | Opis                                                                   |
|----|--------------------------------|------------------------------------------------------------------------|
| U1 | Zmiana nazwiska pacjenta       | Aktualizacja danych pacjenta wskazanego identyfikatorem.               |
| U2 | Zmiana statusu wizyty          | Aktualizacja statusu jednej wizyty wskazanej identyfikatorem.          |
| U3 | Zmiana ceny wykonanej usługi   | Aktualizacja ceny usług powiązanych z wybraną wizytą.                  |
| U4 | Aktualizacja notatek diagnozy  | Zmiana notatek diagnoz przypisanych do wybranej wizyty.                |
| U5 | Zmiana numeru licencji lekarza | Aktualizacja numeru licencji jednego lekarza.                          |
| U6 | Zmiana telefonu oddziału       | Aktualizacja telefonu wybranego oddziału szpitalnego.                  |
| D1 | Usunięcie wyniku badania       | Usunięcie pojedynczego wyniku badania bez rekordów potomnych.          |
| D2 | Usunięcie diagnozy             | Usunięcie pojedynczej diagnozy bez rekordów potomnych.                 |
| D3 | Usunięcie pacjenta             | Usunięcie pacjenta wraz z powiązanymi danymi zależnymi.                |
| D4 | Usunięcie wykonanej usługi     | Usunięcie pojedynczej usługi wykonanej podczas wizyty.                 |
| D5 | Usunięcie recepty              | Usunięcie recepty wraz z jej pozycjami.                                |
| D6 | Kaskadowe usunięcie wizyty     | Usunięcie wizyty oraz wszystkich powiązanych z nią danych klinicznych. |

## 6 Metodyka pomiaru

### 6.1 Schemat pomiaru

Każdy z 24 scenariuszy CRUD (po 6 dla tworzenia, odczytu, aktualizacji i usuwania danych) wykonano dla każdej bazy w dwóch wariantach – *bez indeksów* i *z indeksami* – oraz dla trzech skal danych: **500 000**, **1 000 000** oraz **10 000 000** sumarycznych rekordów. Przed pomiarem wykonywany jest jeden przebieg rozgrzewający, a następnie 3 powtórzenia, z których wyliczana jest średnia arytmetyczna oraz odchylenie standardowe (kolumny `Average_Time_Seconds` i `StdDev_Time_Seconds` w `results/results_all.csv`; w kolumnie `Median_Time_Seconds` dostępna jest mediana kontrolna). Wszystkie wartości w niniejszym rozdziale podawane są w milisekundach [ms].

Dla scenariuszy wstawiania i usuwania mierzony jest wyłącznie czas operacji docelowej – odpowiednio wstawienie nowego rekordu oraz usunięcie istniejącego rekordu. Operacja komplementarna potrzebna do utrzymania stałej liczby wierszy między próbami jest wykonywana w osobnej fazie i nie wpływa na raportowany czas.

### 6.2 Środowisko testowe

Pomiary wykonano na pojedynczej maszynie z Windows 11 + WSL2 i Docker Desktop. Klient pomiarowy (Python 3.13) działał natywnie na hoście Windows i łączył się z bazami przez `localhost`. Konfiguracja kontenerów (`docker-compose.yml`) została celowo *zrównana* między PostgreSQL, MySQL i MongoDB: każdy silnik dyskowy ma 4 GB pamięci podręcznej oraz aktywną pełną synchronizację dyskową przy zatwierdzaniu transakcji. Pozwala to porównywać silniki na zbliżonym budżecie pamięci i przy identycznej polityce trwałości transakcji:

Tabela 6: Parametry uruchomieniowe kontenerów (`docker-compose.yml`)

| Baza       | Wersja | Parametry uruchomieniowe                                                                                                                   |
|------------|--------|--------------------------------------------------------------------------------------------------------------------------------------------|
| PostgreSQL | 16     | 4 GB bufora współdzielonego, 64 MB pamięci roboczej, synchroniczne zatwierdzanie transakcji, maksymalnie 2 procesy równoległe na zapytanie |
| MySQL      | 8.0    | 4 GB puli buforów InnoDB, dziennik 512 MB, synchronizacja dziennika przy każdym zatwierdzeniu transakcji, pakiety do 256 MB                |
| MongoDB    | 7      | 4 GB pamięci podręcznej WiredTiger, dziennik operacji aktywny domyślnie                                                                    |
| Redis      | 7      | limit pamięci 10 GB, brak wyrzucania kluczy, persystencja wyłączona                                                                        |

Po wypełnieniu baz danymi oraz po dodaniu indeksów program pomiarowy wywołuje jawne `ANALYZE` (PostgreSQL) i `ANALYZE TABLE` (MySQL), zapewniając aktualność statystyk optymalizatora. W trakcie pojedynczego przebiegu aktywna jest tylko jedna baza naraz.

### 6.3 Ograniczenia i zastrzeżenia interpretacyjne

- **Liczba prób  $n=3$ :** średnia z 3 powtórzeń jest wrażliwa na pojedyncze odstające pomiary (GC pause, cache miss). Różnice  $<5\%$  między bazami należy traktować jako szum pomiarowy; brak możliwości rygorystycznych testów statystycznych typu t-Student/ANOVA (wymagałyby  $n \geq 30$ ).
- **Redis vs bazy dyskowe:** Redis pracuje wyłącznie w RAM, z wyłączoną persystencją – nie wykonuje I/O dyskowych ani nie prowadzi dziennika transakcji. Bezpośrednie porównanie bezwzględnych czasów Redis z PG/MySQL/Mongo ilustruje raczej *różnicę paradygmatów* (RAM vs dysk) niż jakość implementacji.
- **Indeksy w Redis:** Redis nie posiada mechanizmu indeksów – dostęp jest deterministyczny  $O(1)$  lub  $O(N)$  wynikający z typu struktury danych. Z tego powodu Redis jest **wyłączony z wykresów** porównawczych typu „bez/z indeksami”; w tabelach z indeksami pojawia się jako --.
- **Asymetria odczytu dla Redis:** pobranie wizyt pacjenta z danymi lekarza oraz wyliczenie sumy kosztów usług w Redis ograniczają próbkę kluczy do stałej liczby elementów. Bazy SQL i MongoDB wykonują pełne złączenia lub agregacje z limitem 50 rekordów, dlatego przy rosnącej skali ich koszt rośnie, a koszt Redis pozostaje stały.
- **Ograniczenie modelu MongoDB:** generator nakłada limit 200 wizyt na pacjenta, aby pojedynczy dokument z zagnieżdżonymi wizytami nie przekroczył limitu BSON 16 MB. Przy testowanych skalach średnia wynosi 12–13 wizyt/pacjenta (np. 2 mln wizyt na 160 tys. pacjentów przy 10M), więc limit praktycznie nigdy nie jest aktywny i nie wpływa na rozkład danych. MongoDB testowane jest na identycznym wolumenie rekordów co PostgreSQL i MySQL.

## 7 Wyniki testów wydajnościowych

Każda wartość w tabelach to średnia arytmetyczna z 3 prób w milisekundach [ms]; wartość pogrubiona w wierszu oznacza najszybszą bazę dla danej skali i scenariusza. Pełne wyniki (wszystkie skale + odchylenie standardowe + mediana kontrolna) znajdują się w `results/results_all.csv`.

### 7.1 Operacja wstawiania danych

Mierzymy czas pojedynczego wstawienia wiersza; w scenariuszu wstawienia recepty z pozycjami obejmuje to dodanie recepty oraz jej pozycji w jednej transakcji. Każda wartość jest średnią z 3 powtórzeń wyrażoną w milisekundach.

**Czas wykonania bez indeksów**

Tabela 7: Czas operacji wstawiania danych [ms] – bez indeksow. Najlepszy wynik w wierszu pogrubiony.

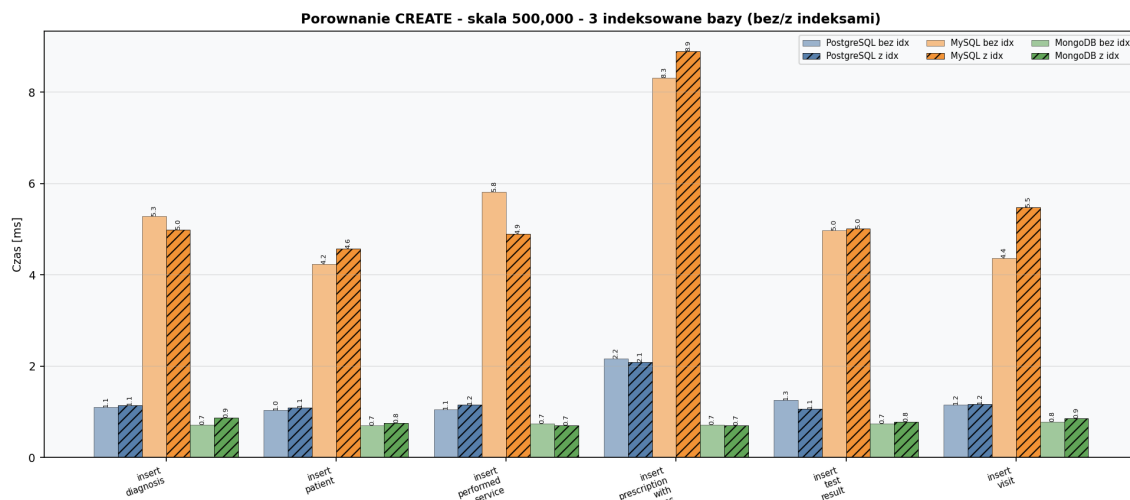
| Scenariusz                     | Skala | PostgreSQL | MySQL | MongoDB | Redis       |
|--------------------------------|-------|------------|-------|---------|-------------|
| Wstawienie diagnozy            | 500k  | 1.10       | 5.28  | 0.72    | <b>0.40</b> |
|                                | 1M    | 1.04       | 5.07  | 0.74    | <b>0.39</b> |
|                                | 10M   | 1.05       | 4.25  | 0.74    | <b>0.40</b> |
| Wstawienie pacjenta            | 500k  | 1.03       | 4.24  | 0.70    | <b>0.46</b> |
|                                | 1M    | 1.12       | 4.21  | 0.69    | <b>0.45</b> |
|                                | 10M   | 1.19       | 3.99  | 0.84    | <b>0.45</b> |
| Wstawienie wykonanej usługi    | 500k  | 1.06       | 5.82  | 0.74    | <b>0.46</b> |
|                                | 1M    | 1.02       | 4.59  | 0.79    | <b>0.53</b> |
|                                | 10M   | 1.06       | 6.81  | 0.85    | <b>0.47</b> |
| Wstawienie recepty z pozycjami | 500k  | 2.16       | 8.31  | 0.72    | <b>0.44</b> |
|                                | 1M    | 2.44       | 8.70  | 0.72    | <b>0.43</b> |
|                                | 10M   | 2.28       | 10.97 | 0.75    | <b>0.47</b> |
| Wstawienie wyniku badania      | 500k  | 1.26       | 4.98  | 0.74    | <b>0.49</b> |
|                                | 1M    | 1.03       | 4.46  | 0.71    | <b>0.43</b> |
|                                | 10M   | 1.30       | 7.48  | 0.72    | <b>0.41</b> |
| Wstawienie wizyty              | 500k  | 1.16       | 4.37  | 0.77    | <b>0.54</b> |
|                                | 1M    | 1.13       | 3.80  | 0.75    | <b>0.44</b> |
|                                | 10M   | 1.25       | 4.45  | 0.93    | <b>0.48</b> |

## Czas wykonania z indeksami

Tabela 8: Czas operacji wstawiania danych [ms] – z indeksami. Najlepszy wynik w wierszu pogrubiony. Redis nie posiada mechanizmu indeksów (kolumna ‘—’).

| Scenariusz                     | Skala | PostgreSQL | MySQL | MongoDB     | Redis |
|--------------------------------|-------|------------|-------|-------------|-------|
| Wstawienie diagnozy            | 500k  | 1.14       | 4.98  | <b>0.87</b> | —     |
|                                | 1M    | 1.12       | 4.38  | <b>0.81</b> | —     |
|                                | 10M   | 1.11       | 4.99  | <b>0.83</b> | —     |
| Wstawienie pacjenta            | 500k  | 1.09       | 4.57  | <b>0.76</b> | —     |
|                                | 1M    | 1.04       | 4.34  | <b>0.68</b> | —     |
|                                | 10M   | 1.06       | 4.23  | <b>0.70</b> | —     |
| Wstawienie wykonanej usługi    | 500k  | 1.16       | 4.89  | <b>0.70</b> | —     |
|                                | 1M    | 1.08       | 4.53  | <b>0.72</b> | —     |
|                                | 10M   | 1.10       | 4.15  | <b>0.82</b> | —     |
| Wstawienie recepty z pozycjami | 500k  | 2.09       | 8.89  | <b>0.70</b> | —     |
|                                | 1M    | 2.05       | 8.57  | <b>0.81</b> | —     |
|                                | 10M   | 2.10       | 7.70  | <b>0.73</b> | —     |
| Wstawienie wyniku badania      | 500k  | 1.06       | 5.01  | <b>0.77</b> | —     |
|                                | 1M    | 1.04       | 5.46  | <b>0.72</b> | —     |
|                                | 10M   | 1.15       | 4.15  | <b>0.73</b> | —     |
| Wstawienie wizyty              | 500k  | 1.17       | 5.47  | <b>0.85</b> | —     |
|                                | 1M    | 1.07       | 4.79  | <b>0.92</b> | —     |
|                                | 10M   | 1.10       | 4.40  | <b>0.87</b> | —     |

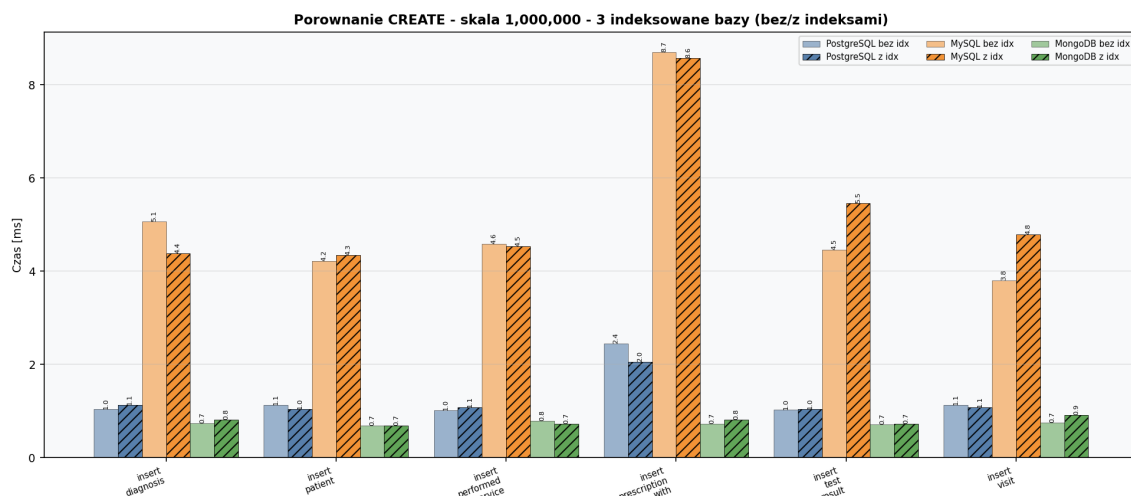
## Porównanie baz na trzech skalach



Rysunek 2: Wstawianie danych – skala 500 000. Każda baza ma parę wyników: wariant bez indeksów oraz wariant z indeksami.

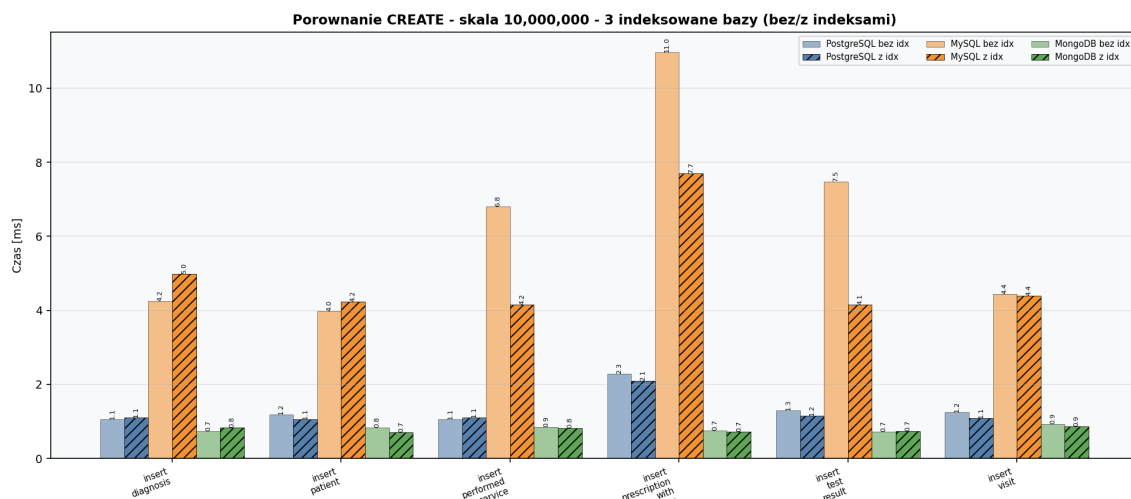
**Interpretacja wyników.** Na skali 500 000 rekordów czasy wstawiania są krótkie i mało zależne od obecności indeksów. MongoDB pozostaje na poziomie około 0.7–0.9 ms, PostgreSQL najczęściej 1–2 ms, a MySQL 4–9 ms. Najwolniejsze jest wstawienie recepty z pozycjami, ponieważ obejmuje kilka powiązanych rekordów w jednej operacji logicznej. Dane

nie pokazują jednak prostego schematu „indeksy spowalniają wstawianie”: różnice między wariantami są niewielkie, czasem korzystne, czasem niekorzystne, a dla MongoDB i PostgreSQL mieszczą się blisko szumu pomiarowego.



Rysunek 3: Wstawianie danych – skala 1 000 000.

**Interpretacja wyników.** Przy 1 000 000 rekordów układ wyników jest analogiczny do skali 500 000: wzrost wolumenu danych nie powoduje systematycznego wzrostu czasu pojedynczego wstawienia. Ważniejsze od skali okazują się cechy silnika i koszt trwałego zapisu. MySQL pozostaje wyraźnie wolniejszy od pozostałych systemów, natomiast MongoDB utrzymuje najniższe czasy. Wariant z indeksami nie tworzy jednolitego trendu: dla jednych scenariuszy jest minimalnie szybszy, dla innych minimalnie wolniejszy.



Rysunek 4: Wstawianie danych – skala 10 000 000.

**Interpretacja wyników.** Na skali 10 000 000 rekordów nadal nie widać liniowego wzrostu czasu wstawiania. Najważniejszym wyjątkiem jest MySQL przy wstawieniu recepty z pozycjami: wariant z indeksami skraca czas z 10.97 ms do 7.70 ms. Nie jest to dowód, że indeksy ogólnie przyspieszają zapis, lecz efekt konkretnego scenariusza, w którym

sprawdzanie powiązań między receptą i pozycjami recepty ma zauważalny koszt. Dla pozostałych przypadków wyniki są mieszane i nie uzasadniają tezy o stałym spowolnieniu ani stałym przyspieszeniu wstawiania.

## Analiza planów wykonania – wstawianie danych

Plany wykonania wstawiania są proste we wszystkich silnikach SQL: pojedyncza operacja zapisu do tabeli poprzedzona ewentualnym sprawdzeniem kluczy obcych. Na poziomie planu wykonania nie ma więc złożonego wyboru ścieżki dostępu – przy wstawianiu danych liczy się przede wszystkim koszt fizycznego zapisu, a nie sposób wyszukiwania rekordów. Realny koszt każdego wstawienia to suma:

- zapisu wiersza do struktury tabeli (drzewo klastrowane w InnoDB, sterta w PostgreSQL, dokument BSON w MongoDB, struktura haszująca w Redis),
- dopisania nowego klucza do każdego istniejącego indeksu wtórnego,
- zapisu do dziennika transakcji (WAL w PostgreSQL, dziennik odtwarzania w MySQL, dziennik operacji w MongoDB),
- synchronizacji dyskowej przy zatwierdzeniu transakcji – w PostgreSQL zbiorowej dla wielu transakcji, w MySQL pojedynczej dla każdej, zgodnie z konfiguracją z Tabeli 6,
- walidacji kluczy obcych po stronie tabeli docelowej (wymaga indeksu na referowanej kolumnie – w MySQL zawsze istnieje, w PG tylko jeśli go utworzymy).

Większość obserwowanej różnicy między bazami pochodzi z dwóch ostatnich pozycji: MySQL płaci za pełną synchronizację przy każdym zatwierdzeniu transakcji, dlatego jest najwolniejszy, a MongoDB i Redis mają łżejszą politykę trwałości. Szczegółową analizę narzutu indeksów na zapis przedstawia współczynnik  $K_{zapis}$  w Sekcji 9.2.

## Wnioski – operacje wstawiania danych

- **Wstawianie danych nie potwierdza jednoznacznie części hipotezy o spowolnieniu zapisu.** W badanych danych wpływ indeksów jest mały albo mieszany; nie pojawia się regularny wzrost czasu wraz ze skalą.
- **Różnice między silnikami są większe niż różnice między wariantami indeksowania:** MongoDB ma najniższe czasy, PostgreSQL pozostaje stabilny w zakresie około 1–2 ms, a MySQL jest najwolniejszy przez koszt trwałego zatwierdzania transakcji.
- **Wstawienie recepty z pozycjami jest najcięższym scenariuszem w bazach relacyjnych.** Wynika to z kilku zapisów i sprawdzenia powiązań, a nie z samego rozmiaru bazy.
- **Wniosek praktyczny:** dla obciążeń zdominowanych przez dopisywanie danych ważniejsza jest konfiguracja trwałości, liczba rekordów zapisywanych w jednej operacji i sposób obsługi powiązań niż sama obecność indeksów dodatkowych.

## 7.2 Operacja odczytu danych

Mierzymy czas pojedynczego odczytu danych: od prostego pobrania rekordu po identyfikację po złączeniu kilku tabel z agregacją. Każda wartość to średnia z 3 powtórzeń [ms].

**Czas wykonania bez indeksów**

Tabela 9: Czas operacji odczytu danych [ms] – bez indeksów. Najlepszy wynik w wierszu pogrubiony.

| Scenariusz                           | Skala | PostgreSQL  | MySQL | MongoDB     | Redis       |
|--------------------------------------|-------|-------------|-------|-------------|-------------|
| Suma kosztów usług pacjenta          | 500k  | 8.50        | 1.41  | 0.75        | <b>0.69</b> |
|                                      | 1M    | 13.55       | 1.39  | 0.77        | <b>0.62</b> |
|                                      | 10M   | 84.85       | 1.50  | 0.76        | <b>0.56</b> |
| Pobranie pacjenta po identyfikatorze | 500k  | <b>0.41</b> | 1.23  | 0.77        | 0.46        |
|                                      | 1M    | 0.41        | 1.25  | 0.77        | <b>0.41</b> |
|                                      | 10M   | 0.48        | 1.35  | 0.89        | <b>0.42</b> |
| Pełna historia pacjenta              | 500k  | 8.58        | 1.46  | <b>0.77</b> | 1.06        |
|                                      | 1M    | 13.85       | 1.42  | <b>0.73</b> | 0.83        |
|                                      | 10M   | 84.63       | 1.94  | <b>0.79</b> | 0.83        |
| Recepty z lekami                     | 500k  | 4.27        | 1.29  | 0.83        | <b>0.16</b> |
|                                      | 1M    | 5.55        | 1.29  | 0.75        | <b>0.00</b> |
|                                      | 10M   | 49.29       | 1.23  | 0.86        | <b>0.44</b> |
| Diagnozy wizyty                      | 500k  | 3.65        | 1.22  | 0.85        | <b>0.17</b> |
|                                      | 1M    | 6.88        | 1.17  | 0.85        | <b>0.28</b> |
|                                      | 10M   | 46.25       | 1.20  | 0.96        | <b>0.28</b> |
| Wizyty pacjenta z lekarzem           | 500k  | 3.80        | 1.49  | 1.71        | <b>1.35</b> |
|                                      | 1M    | 8.97        | 1.40  | 2.02        | <b>1.26</b> |
|                                      | 10M   | 45.48       | 1.47  | 9.21        | <b>1.28</b> |

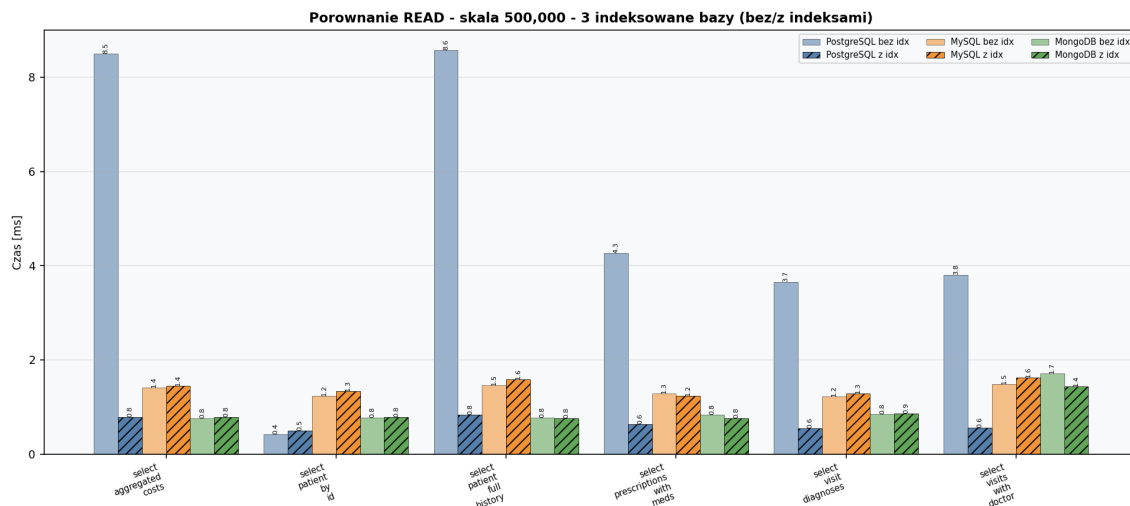
## Czas wykonania z indeksami



Tabela 10: Czas operacji odczytu danych [ms] – z indeksami. Najlepszy wynik w wierszu pogrubiony. Redis nie posiada mechanizmu indeksów (kolumna ‘—’).

| Scenariusz                           | Skala | PostgreSQL  | MySQL | MongoDB     | Redis |
|--------------------------------------|-------|-------------|-------|-------------|-------|
| Suma kosztów usług pacjenta          | 500k  | 0.79        | 1.44  | <b>0.78</b> | —     |
|                                      | 1M    | <b>0.71</b> | 1.36  | 0.92        | —     |
|                                      | 10M   | 1.32        | 1.41  | <b>0.80</b> | —     |
| Pobranie pacjenta po identyfikatorze | 500k  | <b>0.49</b> | 1.33  | 0.79        | —     |
|                                      | 1M    | <b>0.41</b> | 1.28  | 0.80        | —     |
|                                      | 10M   | <b>0.43</b> | 1.24  | 0.89        | —     |
| Pełna historia pacjenta              | 500k  | 0.83        | 1.59  | <b>0.76</b> | —     |
|                                      | 1M    | 1.01        | 1.62  | <b>0.97</b> | —     |
|                                      | 10M   | 1.45        | 1.67  | <b>0.77</b> | —     |
| Recepty z lekami                     | 500k  | <b>0.64</b> | 1.23  | 0.75        | —     |
|                                      | 1M    | <b>0.56</b> | 1.31  | 0.98        | —     |
|                                      | 10M   | <b>0.68</b> | 1.54  | 0.80        | —     |
| Diagnozy wizyty                      | 500k  | <b>0.55</b> | 1.28  | 0.85        | —     |
|                                      | 1M    | <b>0.48</b> | 1.28  | 1.57        | —     |
|                                      | 10M   | <b>0.76</b> | 1.26  | 0.89        | —     |
| Wizyty pacjenta z lekarzem           | 500k  | <b>0.56</b> | 1.62  | 1.43        | —     |
|                                      | 1M    | <b>0.59</b> | 1.51  | 1.50        | —     |
|                                      | 10M   | <b>0.69</b> | 1.50  | 1.55        | —     |

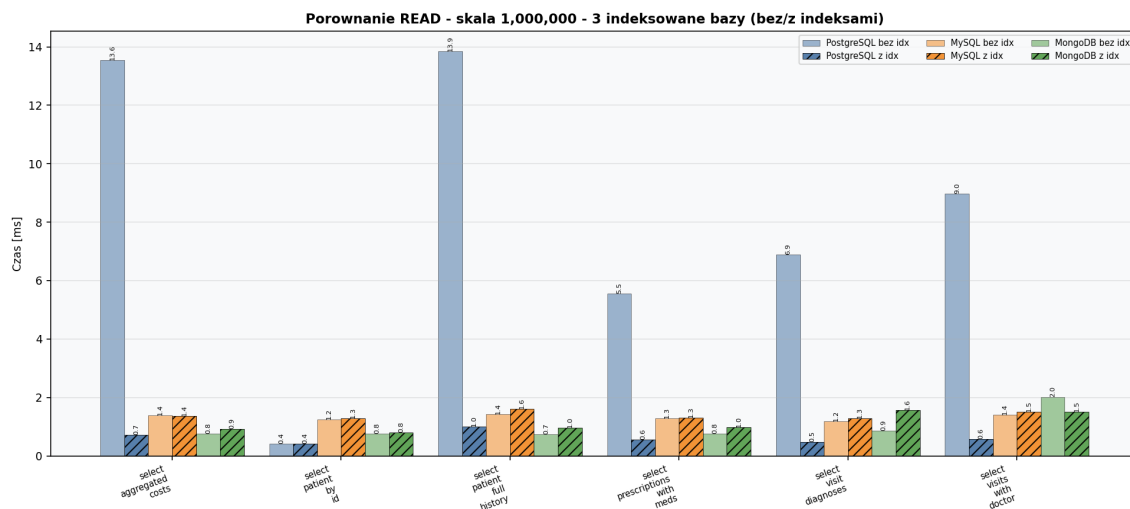
## Porównanie baz na trzech skalach



Rysunek 5: Odczyt danych – skala 500 000.

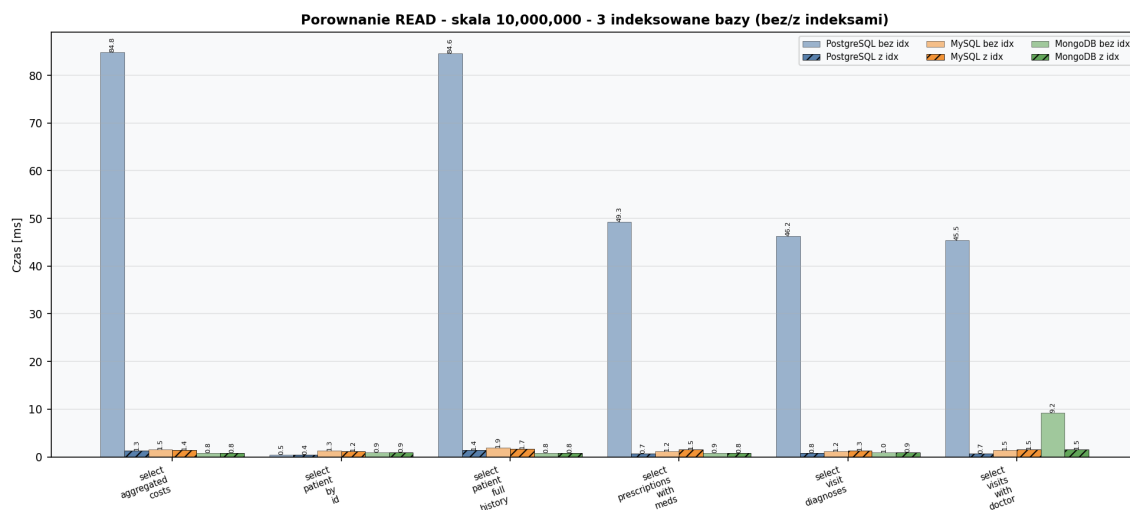
**Interpretacja wyników.** Na skali 500 000 rekordów indeksy wyraźnie pomagają głównie PostgreSQL: w scenariuszach wymagających odszukania danych po kolumnach powiązań czas spada z około 3.6–8.6 ms do około 0.5–0.8 ms. W MySQL nie widać analogicznego zysku, ponieważ czasy pozostają w zakresie około 1.2–1.6 ms, a wariant z indeksami bywa

nawet minimalnie wolniejszy. MongoDB również nie przyspiesza równomiernie: większość odczytów jest już szybka dzięki dostępowi po identyfikatorze dokumentu.



Rysunek 6: Odczyt danych – skala 1 000 000.

**Interpretacja wyników.** Przy 1 000 000 rekordów powtarza się ten sam trend, ale bez potrzeby powtarzania całego mechanizmu: PostgreSQL bez indeksów rośnie do około 6–14 ms w cięższych odczytach, a po indeksowaniu pozostaje zwykle poniżej 1 ms. MySQL nadal nie pokazuje poprawy wynikającej z dodatkowych indeksów, a MongoDB zachowuje się mieszanie: część wyników jest podobna, część nieco gorsza lub lepsza. Odczyt po identyfikatorze pacjenta jest szybki we wszystkich skalach, ponieważ nie wymaga przeszukiwania dużego zbioru rekordów.



Rysunek 7: Odczyt danych – skala 10 000 000. PostgreSQL bez indeksów osiąga 45–85 ms, natomiast po zastosowaniu indeksów 0.6–1.5 ms.

**Interpretacja wyników.** Na skali 10 000 000 rekordów różnice są najbardziej czytelne. PostgreSQL bez indeksów osiąga 45–85 ms w odczytach zależnych od powiązań, a po

indeksowaniu wraca do około 0.6–1.5 ms. MySQL nadal utrzymuje stabilne, ale nie najlepsze czasy około 1.2–1.7 ms; indeksy dodatkowe nie poprawiają tu wyników w sposób mierzalny. W MongoDB znacząca poprawa pojawia się przede wszystkim przy pobraniu wizyt pacjenta z lekarzem (9.21 ms bez indeksów wobec 1.55 ms z indeksem). Pozostałe odczyty dokumentowe pozostają szybkie niezależnie od dodatkowych indeksów.

## Analiza planów wykonania – odczyt danych

**PostgreSQL – pobranie diagnoz wizyty przy 10M rekordów.** Z indeksem na identyfikator wizyty w tabeli diagnoz plan wykorzystuje złączenie haszujące oraz skanowanie indeksu po stronie tabeli diagnoz (4 bloki bufora, czas wykonania **0.21 ms**). Bez indeksu optymalizator wybiera równoległe sekwencyjne przeszukiwanie tabeli z 2 procesami roboczymi, odczytuje 17 539 bloków bufora i filtruje 667 346 wierszy w każdym procesie – czas wykonania wzrasta do **33.5 ms** (160× na poziomie planu). Pełny listing znajduje się w Sekcji 8.2.

**PostgreSQL – wyliczenie sumy kosztów usług przy 500k rekordów.** Z indeksami silnik wykonuje złączenie pętlowe z dwoma skanowaniami indeksu (39 bloków bufora, 0.59 ms). Bez indeksów wykorzystuje złączenie haszujące z dwoma sekwencyjnymi przeglądami tabel (1374 bloki, 13.0 ms). Daje to **22-krotne przyspieszenie na poziomie planu** dla skali 500k; w pomiarze właściwym uzyskano  $S_{odczyt} = 10.79\times$  – różnica wynika ze stałego narzutu sterownika Python+psycopg, który dominuje nad submilisekundowym planem.

**MySQL – pobranie diagnoz wizyty.** Analiza planu pokazuje wyszukanie po indeksie na identyfikatorze wizyty oraz pobranie pojedynczego wiersza po kluczu głównym – całość trwa 0.01 ms na poziomie silnika. Czas zaobserwowany w pomiarze właściwym to 1.23 ms, czyli sam silnik odpowiada za około 1% czasu, a reszta to protokół klient-serwer i sterownik. To tłumaczy, dlaczego włączanie lub wyłączanie indeksów w MySQL nie zmienia istotnie wyniku: różnica między 0.01 ms a 0.5 ms znika przy około 1 ms narzutu komunikacyjnego.

**MongoDB – pobranie wizyt pacjenta z lekarzem.** Z indeksem na identyfikator lekarza plan wykonuje skanowanie indeksu i pobranie pasujących dokumentów, badając około 10 dokumentów w około 1 ms. Bez indeksu silnik wykonuje pełny przegląd kolekcji, badając 2 000 000 dokumentów w około 9 ms. MongoDB nie używa tu równoległości tak jak PostgreSQL, więc skala pełnego przeglądu kolekcji przekłada się bezpośrednio na czas wykonania.

## Wnioski – operacje odczytu danych

- **Teza o przyspieszeniu odczytu jest silnie potwierdzona dla PostgreSQL.** Efekt rośnie wraz ze skalą i przy 10M rekordów osiąga około 59–72-krotność w scenariuszach opartych na powiązaniach między tabelami.
- **W MySQL dodatkowe indeksy nie dają dużego zysku, ale nie obala to tezy o znaczeniu indeksów.** W tym schemacie InnoDB tworzy indeksy wymagane przez klucze obce, więc wariant „bez indeksów” nadal korzysta z części najważniejszych struktur dostępu. Dlatego czasy odczytu pozostają stabilne, a różnica między wariantami jest mała.
- **W MongoDB wpływ indeksów jest selektywny.** Wyraźny zysk pojawia się przy pobraniu wizyt pacjenta z lekarzem na największej skali; odczyty po identyfikatorze dokumentu albo po danych już dobrze osadzonych w dokumencie pozostają

szybkie niezależnie od dodatkowych indeksów.

- **Wniosek praktyczny:** indeksy należy oceniać w kontekście silnika i modelu danych. W PostgreSQL indeksowanie kolumn powiązań jest konieczne dla dużych tabel, w MySQL efekt może być niewidoczny przy automatycznych indeksach i dobrym buforowaniu, a w MongoDB indeks powinien odpowiadać faktycznemu polu filtrowania.

### 7.3 Operacja aktualizacji danych

Mierzmy czas pojedynczej aktualizacji danych – od najprostszej zmiany statusu jednej wizyty po klucz głównym po aktualizację, w której warunek filtrowania odnosi się do kolumny klucza obcego i obejmuje kilka rekordów. Każda wartość to średnia z 3 powtórzeń [ms].

#### Czas wykonania bez indeksów

Tabela 11: Czas operacji aktualizacji danych [ms] – bez indeksów. Najlepszy wynik w wierszu pogrubiony.

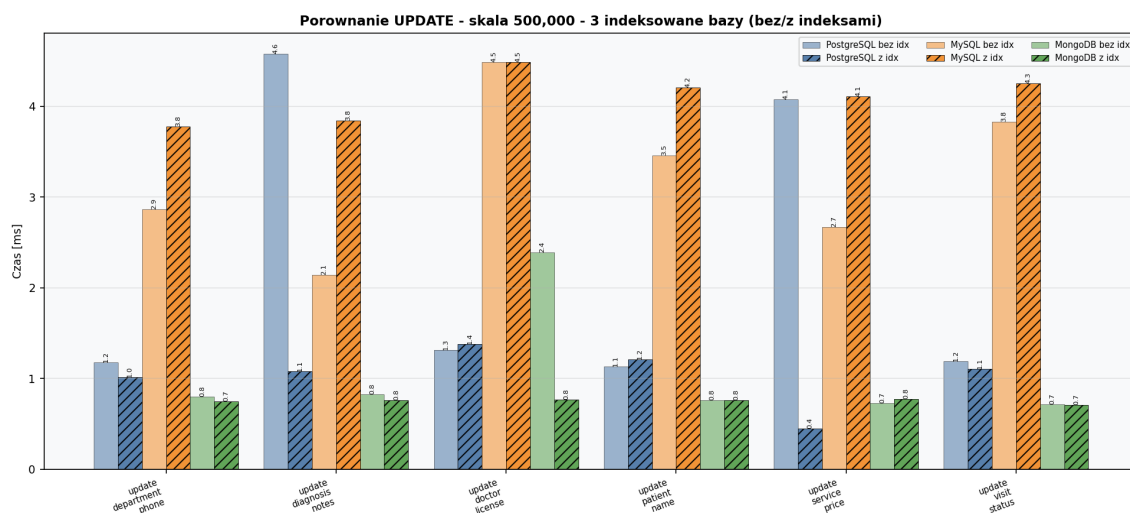
| Scenariusz                    | Skala | PostgreSQL | MySQL | MongoDB     | Redis       |
|-------------------------------|-------|------------|-------|-------------|-------------|
| Zmiana telefonu oddziału      | 500k  | 1.18       | 2.86  | 0.80        | <b>0.39</b> |
|                               | 1M    | 0.94       | 5.40  | 0.76        | <b>0.40</b> |
|                               | 10M   | 1.06       | 3.99  | 0.75        | <b>0.39</b> |
| Aktualizacja notatek diagnozy | 500k  | 4.58       | 2.14  | <b>0.82</b> | 1.03        |
|                               | 1M    | 7.10       | 3.72  | <b>0.72</b> | 0.81        |
|                               | 10M   | 97.07      | 2.80  | <b>0.71</b> | 0.82        |
| Zmiana licencji lekarza       | 500k  | 1.32       | 4.49  | 2.39        | <b>0.42</b> |
|                               | 1M    | 0.94       | 4.96  | 0.75        | <b>0.41</b> |
|                               | 10M   | 1.14       | 3.91  | 0.81        | <b>0.39</b> |
| Zmiana nazwiska pacjenta      | 500k  | 1.13       | 3.46  | 0.76        | <b>0.44</b> |
|                               | 1M    | 1.09       | 3.89  | 0.75        | <b>0.44</b> |
|                               | 10M   | 1.07       | 3.90  | 0.79        | <b>0.44</b> |
| Zmiana ceny wykonanej usługi  | 500k  | 4.08       | 2.67  | 0.73        | <b>0.48</b> |
|                               | 1M    | 7.05       | 6.01  | 0.71        | <b>0.47</b> |
|                               | 10M   | 67.45      | 5.37  | 0.71        | <b>0.41</b> |
| Zmiana statusu wizyty         | 500k  | 1.19       | 3.83  | 0.72        | <b>0.51</b> |
|                               | 1M    | 0.94       | 2.07  | 0.72        | <b>0.38</b> |
|                               | 10M   | 1.03       | 2.06  | <b>0.69</b> | 0.83        |

#### Czas wykonania z indeksami

Tabela 12: Czas operacji aktualizacji danych [ms] – z indeksami. Najlepszy wynik w wierszu pogrubiony. Redis nie posiada mechanizmu indeksów (kolumna ‘—’).

| Scenariusz                    | Skala | PostgreSQL  | MySQL | MongoDB     | Redis |
|-------------------------------|-------|-------------|-------|-------------|-------|
| Zmiana telefonu oddziału      | 500k  | 1.02        | 3.78  | <b>0.75</b> | —     |
|                               | 1M    | 1.25        | 4.74  | <b>0.71</b> | —     |
|                               | 10M   | <b>1.29</b> | 4.05  | 1.47        | —     |
| Aktualizacja notatek diagnozy | 500k  | 1.08        | 3.85  | <b>0.76</b> | —     |
|                               | 1M    | 1.03        | 2.24  | <b>0.72</b> | —     |
|                               | 10M   | 1.54        | 2.45  | <b>0.74</b> | —     |
| Zmiana licencji lekarza       | 500k  | 1.38        | 4.49  | <b>0.77</b> | —     |
|                               | 1M    | 1.00        | 3.99  | <b>0.82</b> | —     |
|                               | 10M   | 1.09        | 3.40  | <b>0.76</b> | —     |
| Zmiana nazwiska pacjenta      | 500k  | 1.21        | 4.21  | <b>0.76</b> | —     |
|                               | 1M    | 1.03        | 4.43  | <b>0.84</b> | —     |
|                               | 10M   | 1.50        | 4.74  | <b>0.89</b> | —     |
| Zmiana ceny wykonanej usługi  | 500k  | <b>0.45</b> | 4.11  | 0.78        | —     |
|                               | 1M    | <b>0.63</b> | 3.54  | 0.72        | —     |
|                               | 10M   | 0.94        | 4.28  | <b>0.74</b> | —     |
| Zmiana statusu wizyty         | 500k  | 1.11        | 4.25  | <b>0.71</b> | —     |
|                               | 1M    | 0.99        | 3.23  | <b>0.77</b> | —     |
|                               | 10M   | 1.28        | 3.44  | <b>0.80</b> | —     |

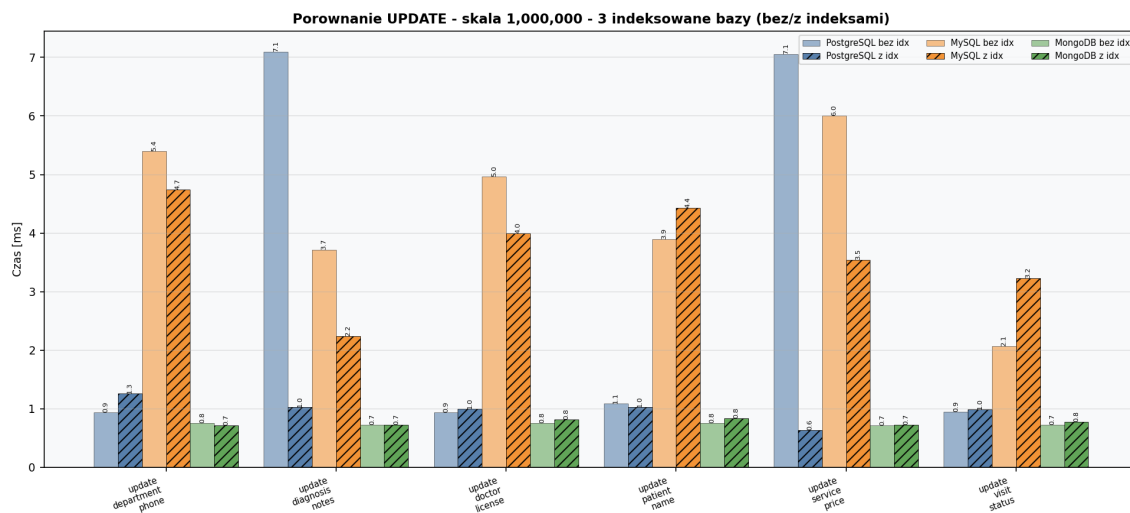
## Porównanie baz na trzech skalach



Rysunek 8: Aktualizacja danych – skala 500 000.

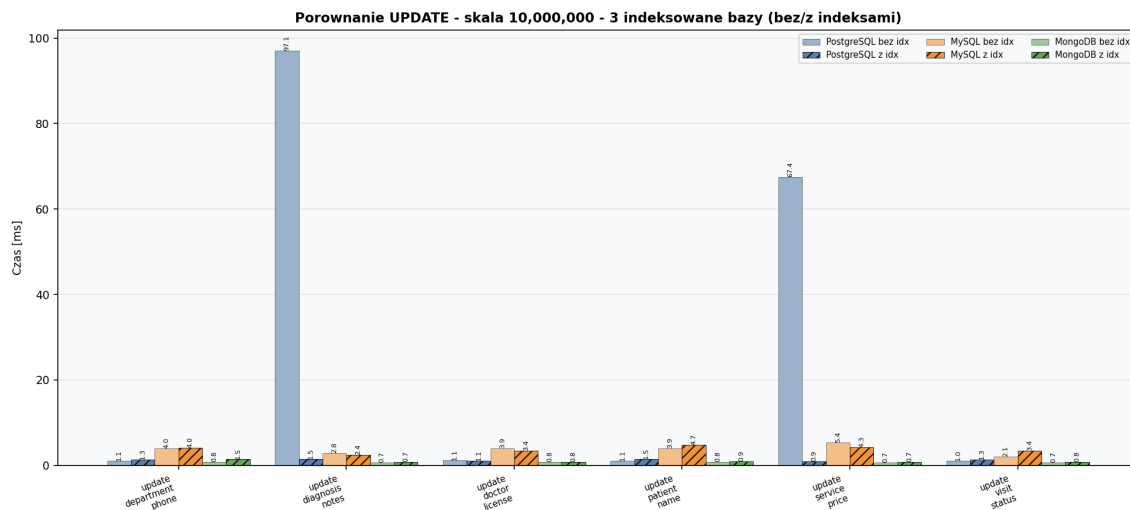
**Interpretacja wyników.** Na skali 500 000 rekordów większość aktualizacji mieści się w kilku milisekundach. MongoDB jest zwykle najszybsza, MySQL ma wyraźnie wyższy koszt zapisu, a PostgreSQL zachowuje się dwojako. Przy zmianie ceny wykonanej usługi i aktualizacji notatek diagnozy indeksy obniżają czas z około 4–5 ms do około 1 ms, ponieważ pomagają znaleźć rekordy powiązane z wizytą. W aktualizacjach po identyfikatorze,

takich jak zmiana nazwiska pacjenta lub statusu wizyty, zysk z wyszukiwania już istnieje dzięki kluczowi głównemu, więc dodatkowe indeksy mogą być wyłącznie narzutem.



Rysunek 9: Aktualizacja danych – skala 1 000 000.

**Interpretacja wyników.** Przy 1 000 000 rekordów nie pojawia się nowy mechanizm, tylko pogłębia się różnica w PostgreSQL dla aktualizacji wyszukujących dane po kolumnie powiązania: bez indeksów czas rośnie do około 7 ms, a z indeksami pozostaje blisko 0.6–1.0 ms. MySQL i MongoDB nie pokazują takiej zależności od skali; ich wyniki są raczej stabilne albo mieszane, co wskazuje, że w tych silnikach koszt pojedynczej aktualizacji jest w tym teście zdominowany przez inne elementy niż sam rozmiar zbioru.



Rysunek 10: Aktualizacja danych – skala 10 000 000. PostgreSQL bez indeksów dla zmiany ceny wykonanej usługi i aktualizacji notatek diagnozy osiąga 67–97 ms.

**Interpretacja wyników.** Na skali 10 000 000 rekordów różnica w PostgreSQL staje się bardzo duża: zmiana ceny wykonanej usługi oraz aktualizacja notatek diagnozy bez indeksów osiągają 67–97 ms, a z indeksami około 1 ms. Nie oznacza to, że indeksy zawsze przyspieszają zapis; oznacza to, że w tych dwóch scenariuszach koszt znalezienia rekordów

jest większy niż koszt samej modyfikacji. Jednocześnie zmiana statusu wizyty w PostgreSQL zwalnia po dodaniu indeksów z 1.03 do 1.28 ms, a w MySQL z 2.06 do 3.44 ms. Tabele pokazują więc dwa zjawiska naraz: indeks może dramatycznie pomóc, gdy jest potrzebny do znalezienia rekordów, ale może spowolnić aktualizację po kluczu głównym.

## Analiza planów wykonania – aktualizacja danych

**PostgreSQL – zmiana ceny wykonanej usługi przy 10M rekordów.** Z indeksem na identyfikator wizyty w tabeli wykonanych usług plan aktualizacji poprzedzony jest skanowaniem indeksu (4 bloki bufora, czas wykonania **0.09 ms**). Bez indeksu silnik wykonuje sekwencyjne przeszukiwanie tabeli z filtrem po identyfikatorze wizyty, odrzucając 2000 195 wierszy (12 741 bloków bufora, czas wykonania **63.0 ms**). To około 700-krotne przyspieszenie na poziomie planu i tłumaczy bezpośrednio, dlaczego  $K_{zapis} = 0.02$  dla tego scenariusza – koszt wyszukania kilku wierszy do aktualizacji *dominuje* nad narzutem aktualizacji indeksów, więc dodanie indeksów daje netto przyspieszenie zapisu.

**PostgreSQL – zmiana statusu wizyty.** Tu plan wygląda identycznie z i bez indeksów dodatkowych: aktualizacja wizyty poprzedzona jest skanowaniem indeksu po kluczu głównym tabeli wizyt (jednoznaczny dostęp, <0.1 ms). Różnicę robi tylko narzut aktualizacji indeksów wtórnych – przy każdej aktualizacji wiersza trzeba przepisać wpisy w dodatkowych indeksach dotyczących lekarza, statusu, daty wizyty itp. Stąd wariant „z indeksami” jest około 24% wolniejszy.

**MySQL – zmiana statusu wizyty.** Interpretacja jest taka sama jak w PostgreSQL, ale efekt jest silniejszy (+67% przy 10M) – każdy zapis musi dodatkowo zaktualizować dziennik odtwarzania i dziennik binarny oraz wymusić synchronizację dyskową, więc każdy dodatkowy indeks dokłada koszt nie tylko aktualizacji struktury indeksu, ale też wpisu do dziennika.

## Wnioski – operacje aktualizacji danych

- **Aktualizacje nie potwierdzają prostej wersji hipotezy.** Nie można powiedzieć, że indeksy po prostu spowalniają zapis: w PostgreSQL dla aktualizacji po kolumnie powiązania indeksy przyspieszają operację kilkadziesiąt razy.
- **Spowolnienie zapisu jest widoczne tam, gdzie rekord i tak jest odnajdywany po kluczu głównym.** Przykładem jest zmiana statusu wizyty: przy 10M rekordów PostgreSQL zwalnia o około 24%, a MySQL o około 67%.
- **MySQL ma stabilne, ale relatywnie wysokie czasy aktualizacji.** Wynika to z kosztu trwałego zatwierdzania zmian; dodatkowe indeksy często zwiększają koszt, choć nie w każdym scenariuszu tak samo.
- **MongoDB osiąga najniższe czasy aktualizacji** spośród baz biorących udział w badaniu indeksów, ponieważ operuje głównie po identyfikatorach dokumentów. Dodatkowe indeksy nie tworzą tu silnego, regularnego trendu.
- **Wniosek praktyczny:** indeksuj kolumny używane do wyszukania aktualizowanych rekordów, ale nie traktuj każdego indeksu jako darmowego. Jeżeli aktualizacja dotyczy często zmienianej kolumny i rekord jest już odnajdywany po kluczu głównym, dodatkowy indeks może być czystym kosztem.

## 7.4 Operacja usuwania danych

Mierzmy czas pojedynczej operacji usuwania danych – od prostych „liści”, takich jak usunięcie wyniku badania bez rekordów potomnych, po kaskadowe usunięcie wizyty wraz z jej diagnozami, usługami, lekami i innymi powiązanymi rekordami. Każda wartość to średnia z 3 powtórzeń [ms].

### Czas wykonania bez indeksów

Tabela 13: Czas operacji usuwania danych [ms] – bez indeksów. Najlepszy wynik w wierszu pogrubiony.

| Scenariusz                 | Skala | PostgreSQL | MySQL | MongoDB | Redis       |
|----------------------------|-------|------------|-------|---------|-------------|
| Usunięcie diagnozy         | 500k  | 0.95       | 4.65  | 0.74    | <b>0.43</b> |
|                            | 1M    | 0.93       | 3.84  | 0.70    | <b>0.39</b> |
|                            | 10M   | 1.13       | 4.45  | 0.71    | <b>0.40</b> |
| Usunięcie pacjenta         | 500k  | 4.64       | 5.01  | 0.69    | <b>0.50</b> |
|                            | 1M    | 7.52       | 3.80  | 0.82    | <b>0.49</b> |
|                            | 10M   | 78.57      | 4.05  | 0.69    | <b>0.39</b> |
| Usunięcie wykonanej usługi | 500k  | 1.07       | 4.90  | 0.76    | <b>0.43</b> |
|                            | 1M    | 0.98       | 4.26  | 0.72    | <b>0.38</b> |
|                            | 10M   | 1.09       | 3.84  | 0.72    | <b>0.38</b> |
| Usunięcie recepty          | 500k  | 3.66       | 4.60  | 0.73    | <b>0.51</b> |
|                            | 1M    | 6.24       | 4.09  | 0.82    | <b>0.38</b> |
|                            | 10M   | 61.49      | 4.47  | 0.76    | <b>0.38</b> |
| Usunięcie wyniku badania   | 500k  | 1.02       | 4.34  | 0.74    | <b>0.44</b> |
|                            | 1M    | 0.93       | 4.19  | 0.73    | <b>0.39</b> |
|                            | 10M   | 1.07       | 4.06  | 0.71    | <b>0.40</b> |
| Kaskadowe usunięcie wizyty | 500k  | 12.75      | 12.29 | 0.73    | <b>0.59</b> |
|                            | 1M    | 22.79      | 4.05  | 0.84    | <b>0.39</b> |
|                            | 10M   | 224.6      | 4.23  | 0.73    | <b>0.42</b> |

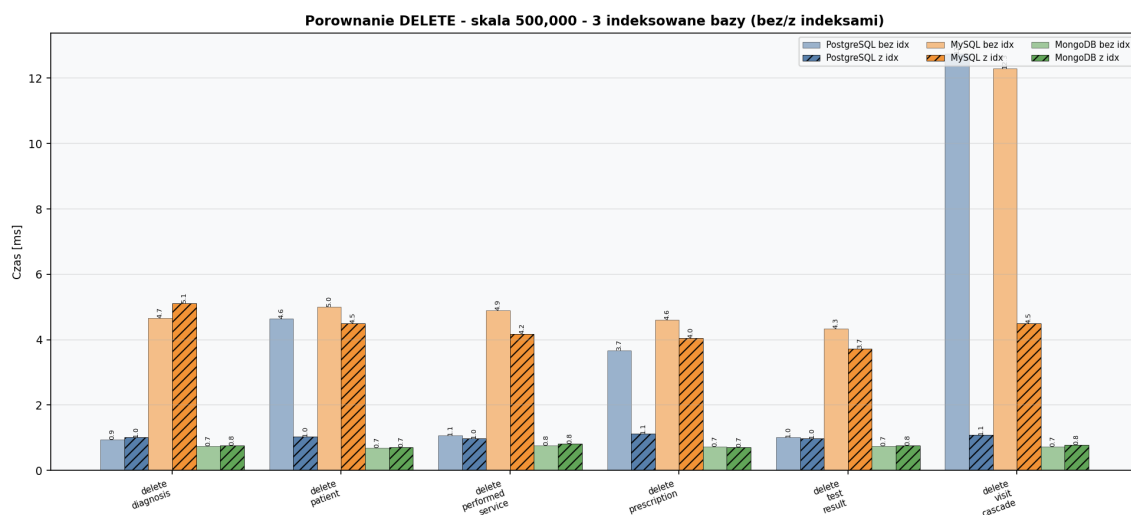
### Czas wykonania z indeksami



Tabela 14: Czas operacji usuwania danych [ms] – z indeksami. Najlepszy wynik w wierszu pogrubiony. Redis nie posiada mechanizmu indeksów (kolumna ‘—’).

| Scenariusz                 | Skala | PostgreSQL | MySQL | MongoDB     | Redis |
|----------------------------|-------|------------|-------|-------------|-------|
| Usunięcie diagnozy         | 500k  | 1.01       | 5.10  | <b>0.76</b> | —     |
|                            | 1M    | 0.95       | 4.24  | <b>0.74</b> | —     |
|                            | 10M   | 0.99       | 4.02  | <b>0.80</b> | —     |
| Usunięcie pacjenta         | 500k  | 1.04       | 4.51  | <b>0.72</b> | —     |
|                            | 1M    | 1.06       | 3.86  | <b>0.73</b> | —     |
|                            | 10M   | 1.03       | 3.82  | <b>0.71</b> | —     |
| Usunięcie wykonanej usługi | 500k  | 0.98       | 4.18  | <b>0.81</b> | —     |
|                            | 1M    | 0.94       | 3.97  | <b>0.78</b> | —     |
|                            | 10M   | 1.02       | 4.12  | <b>0.71</b> | —     |
| Usunięcie recepty          | 500k  | 1.13       | 4.04  | <b>0.71</b> | —     |
|                            | 1M    | 0.98       | 4.47  | <b>0.72</b> | —     |
|                            | 10M   | 1.00       | 3.91  | <b>0.73</b> | —     |
| Usunięcie wyniku badania   | 500k  | 0.99       | 3.73  | <b>0.76</b> | —     |
|                            | 1M    | 1.25       | 3.99  | <b>0.69</b> | —     |
|                            | 10M   | 1.06       | 4.13  | <b>0.71</b> | —     |
| Kaskadowe usunięcie wizyty | 500k  | 1.08       | 4.50  | <b>0.78</b> | —     |
|                            | 1M    | 1.01       | 4.53  | <b>0.77</b> | —     |
|                            | 10M   | 1.05       | 4.43  | <b>0.80</b> | —     |

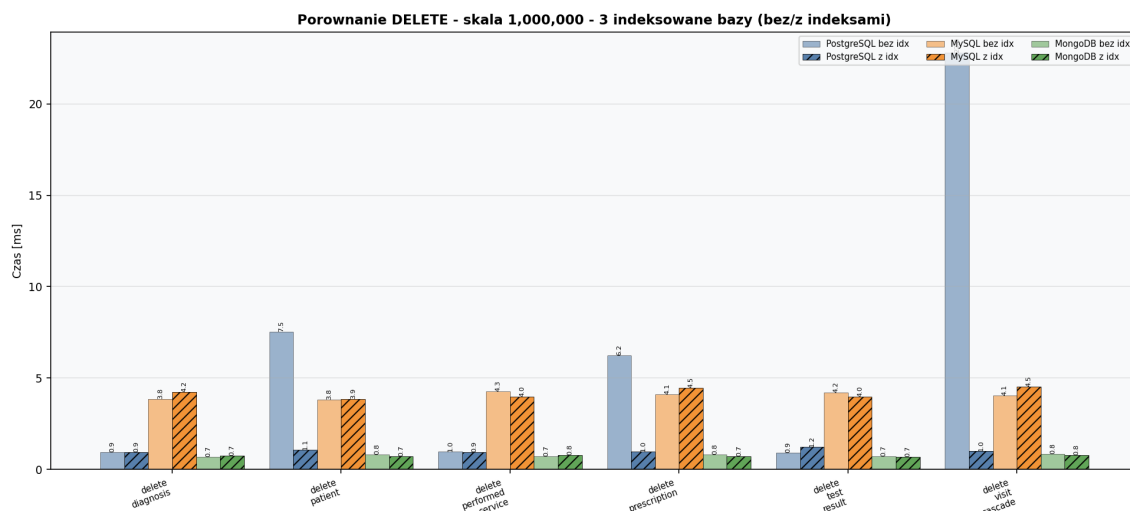
## Porównanie baz na trzech skalach



Rysunek 11: Usuwanie danych – skala 500 000.

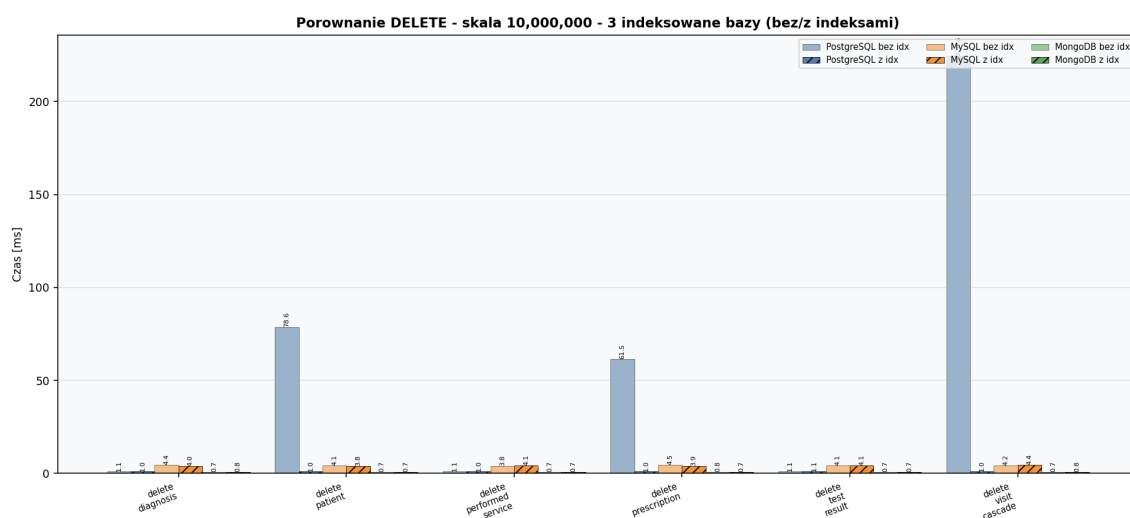
**Interpretacja wyników.** Na skali 500 000 rekordów usuwanie rekordów bez zależnych danych pozostaje krótkie we wszystkich bazach, szczególnie w MongoDB. Różnica pojawia się przy usuwaniu rekordów nadrzędnych, czyli pacjenta, recepty i wizyty. W PostgreSQL bez indeksów kaskadowe usunięcie wizyty trwa około 13 ms, a po indeksowaniu około

1 ms. MySQL ma przy tej skali również wysoki wynik dla kaskadowego usunięcia wizyty bez dodatkowych indeksów, lecz w kolejnych skalach utrzymuje się blisko 4 ms dzięki indeksom wymagany przez mechanizmy integralności.



Rysunek 12: Usuwanie danych – skala 1 000 000.

**Interpretacja wyników.** Przy 1 000 000 rekordów wzorec jest analogiczny: w PostgreSQL bez indeksów rosną przede wszystkim usunięcia rekordów mających zależności, szczególnie wizyta, pacjent i recepta. Po indeksowaniu wszystkie te przypadki wracają w okolice 1 ms. Usunięcia pojedynczych rekordów bez potomków pozostają stabilne, więc problemem nie jest samo usunięcie wiersza, lecz konieczność odnalezienia powiązanych rekordów w tabelach zależnych.



Rysunek 13: Usuwanie danych – skala 10 000 000. PostgreSQL bez indeksów dla kaskadowego usunięcia wizyty osiąga 225 ms; z indeksami spada do 1 ms.

**Interpretacja wyników.** Na skali 10 000 000 rekordów kaskadowe usunięcie wizyty w PostgreSQL bez indeksów osiąga około 225 ms, a z indeksami około 1 ms. Podobny, choć mniejszy, efekt dotyczy usunięcia pacjenta i recepty. Dla rekordów bez potomków indeksy

nie zmieniają wyniku istotnie, bo wystarcza dostęp po kluczu głównym. W MySQL czasy usuwania pozostają blisko 4 ms, a MongoDB utrzymuje około 0.7–0.8 ms, dlatego wniosek o krytycznym znaczeniu indeksów dotyczy tu przede wszystkim PostgreSQL i operacji na rekordach nadrzędnych.

## Analiza planów wykonania – usuwanie danych

**PostgreSQL – kaskadowe usunięcie wizyty przy 10M rekordów.** Z indeksami plan obejmuje usunięcie wizyty po kluczu głównym oraz sześć usunąć w tabelach potomnych z wyszukaniem przez indeks po kolumnie klucza obcego. Łączny czas wykonania wynosi **1.05 ms**. Bez indeksów PostgreSQL nadal odnajduje wizytę po kluczu głównym w czasie poniżej 0.1 ms, ale kaskadowe usuwanie wymaga wyszukania potomków – każda z 6 tabel jest skanowana sekwencyjnie po 2–10 mln wierszy. Pojedynczy sekwencyjny przegląd tabeli wykonanych usług (2 mln wierszy) trwa około 35 ms, więc 6 takich skanów sumarycznie daje około **225 ms**. To około 215-krotne przyspieszenie, największe w całym pomiarze.

**PostgreSQL – usuwanie rekordów bez potomków.** Dla usunięcia wyniku badania, diagnozy lub wykonanej usługi plan jest prosty: usunięcie z tabeli po wyszukaniu wiersza przez klucz główny w czasie około 0.05 ms. Brak potomków oznacza brak dodatkowych skanów. Dodatkowe indeksy nie zmieniają czasu wyszukania, ponieważ klucz główny i tak istnieje, więc ich obecność tworzy tylko niewielki narzut na usunięcie wpisów.

**MySQL.** Kaskadowe usunięcie wizyty korzysta z automatycznych indeksów wymuszonych przez klucze obce – plan wykonuje usunięcie z wyszukaniem po indeksie dla każdej tabeli potomnej. Czasy są stabilne i wynoszą około 4 ms w obu wariantach, ponieważ kluczowe indeksy są zawsze obecne.

**MongoDB.** Wszystkie usunięcia dotyczą pojedynczego dokumentu wyszukiwanego po identyfikatorze, więc plan korzysta z najszybszej ścieżki dostępu po kluczu dokumentu. Brak kaskady w silniku oznacza brak dodatkowych skanów.

## Wnioski – operacje usuwania danych

- **Największy efekt indeksów w całym pomiarze dotyczy PostgreSQL i usuwania rekordów nadrzędnych.** Kaskadowe usunięcie wizyty przy 10M rekordów skraca się z 224.62 do 1.04 ms, czyli około 215-krotnie.
- **Indeksy nie są równie ważne dla każdego usunięcia.** Wynik badania, diagnoza i wykonana usługa są usuwane szybko niezależnie od wariantu, ponieważ nie wymagają sprawdzania dużych tabel potomnych.
- **MySQL i MongoDB nie pokazują takiego samego efektu jak PostgreSQL.** MySQL korzysta z indeksów wymaganych przez integralność relacyjną, a MongoDB usuwa pojedyncze dokumenty po identyfikatorze; dlatego ich czasy są stabilniejsze między skalami.
- **Wniosek praktyczny:** w PostgreSQL kolumny kluczy obcych używane przy usuwaniu kaskadowym muszą być indeksowane ręcznie. W przeciwnym razie usunięcie jednego rekordu nadrzędnego może wymusić wielokrotne przeglądanie wielomilionowych tabel zależnych.

## 8 Analiza indeksów

### 8.1 Zastosowane indeksy

W systemach relacyjnych utworzono 16 indeksów B-tree na kolumnach kluczy obcych i kolumnach filtrujących. W MongoDB utworzono 7 indeksów na kolekcji pacjentów, m.in. dla statusu wizyty, identyfikatora lekarza oraz identyfikatora choroby w zagnieżdżonej diagnozie.

### 8.2 Analiza planów wykonania zapytań

W celu uwiarygodnienia interpretacji wyników pomiaru wykonano dodatkowe analizy planów wykonania zapytań w PostgreSQL, MySQL i MongoDB. Pełne wyniki znajdują się w katalogu results.

#### 8.2.1 PostgreSQL – pobranie diagnoz wizyty (10M, złączenie po kluczu obcym)

Z indeksami:

```
Hash Join (cost=8.49..21.01 rows=2 width=53) (actual time=0.151..0.153 rows=0)
  Buffers: shared hit=1 read=3
  -> Seq Scan on diseases ds (rows=1)
  -> Hash -> Index Scan using idx_diagnoses_visit_id on diagnoses dg
        Index Cond: (visit_id = 1)
        Buffers: shared read=3
Planning Time: 1.221 ms; Execution Time: 0.208 ms
```

Bez indeksów:

```
Gather (cost=1000.28..28974.80 rows=2) (actual time=29.111..33.436)
  Workers Planned: 2; Workers Launched: 2
  Buffers: shared hit=17539
  -> Parallel Seq Scan on diagnoses dg
        Filter: (visit_id = 1); Rows Removed by Filter: 667346
        Buffers: shared hit=17539
Execution Time: 33.454 ms
```

Dla 2 mln diagnoz przy 10M sumarycznych rekordów optymalizator wybiera równoległe sekwencyjne przeszukiwanie tabeli z 2 procesami roboczymi (17 539 trafień w buforze) zamiast zwykłego sekwencyjnego przeglądu. Mimo zrównoleglenia czas wzrasta z 0.2 ms do 33.5 ms ( $\approx 160\times$  w planie; w pomiarze właściwym  $88\times$  ze względu na stały narzut Python+psycopg na pojedynczy pomiar).

#### 8.2.2 PostgreSQL – wyliczenie sumy kosztów usług (500k, kontrolna analiza planu)

Z indeksami:

```
Limit (actual time=... rows=8)
  -> GroupAggregate -> Sort -> Nested Loop
        -> Index Scan using idx_visits_patient_id on visits v
              Index Cond: (patient_id = ?); rows=12
        -> Index Scan using idx_performed_services_visit_id on ps
              Index Cond: (visit_id = v.id); rows=1, loops=12
  Buffers: shared hit=27 read=12
Execution Time: 0.588 ms
```

Bez indeksów:

```

Limit (actual time=... rows=8)
-> Hash Join
    Hash Cond: (ps.visit_id = v.id)
    -> Seq Scan on performed_services ps (rows=100119)
    -> Hash -> Seq Scan on visits v
        Filter: (patient_id = ?); Rows Removed by Filter: 99989
    Buffers: shared hit=1374
Execution Time: 13.005 ms

```

**Wniosek:** kontrolna analiza planu na skali 500k pokazuje **22-krotne przyspieszenie** dzięki indeksom na poziomie planu (13.005 ms  $\rightarrow$  0.588 ms; liczba bloków: 1374  $\rightarrow$  39). W pomiarze właściwym przy 500k uzyskano  $S_{odczyt} = 10.79\times$  – różnica ( $\sim 2\times$ ) wynika ze stałego narzutu Python+psycopg na pojedynczy pomiar, który dominuje przy submilisekundowych planach. Przy 10M analogiczny plan ma czas około 1.3 ms z indeksami oraz około 85 ms bez indeksów ( $\approx 65\times$  w pomiarze właściwym), co potwierdza monotoniczny wzrost efektywności indeksu ze skalą.

### 8.2.3 PostgreSQL – zmiana ceny wykonanej usługi (aktualizacja po kluczu obcym)

Ten przypadek wyjaśnia *przyspieszenie* zapisu z indeksami ( $K_{zapis} = 0.02$ , Sekcja 9.2). **Z indeksem:**

```

Update on performed_services (actual time=0.058..0.058 rows=0)
-> Index Scan using idx_performed_services_visit_id
    Index Cond: (visit_id = 1); Buffers: shared hit=4
Execution Time: 0.088 ms

```

**Bez indeksu:**

```

Update on performed_services (actual time=63.014..63.015 rows=0)
-> Seq Scan on performed_services
    Filter: (visit_id = 1)
    Rows Removed by Filter: 2000195
    Buffers: shared hit=12741
Execution Time: 63.034 ms

```

Koszt znalezienia *kilku* wierszy do zaktualizowania przez indeks dominuje nad narzutem aktualizacji indeksu – stąd  $K_{zapis} \ll 1$  pomimo obecności indeksów.

### 8.2.4 MySQL 8.0 – pobranie diagnoz wizyty

**Z indeksami:**

```

-> Nested loop inner join (cost=0.7 rows=1) (actual time=0.0102..0.0102)
-> Filter: (dg.disease_id is not null)
    -> Index lookup on dg using idx_diagnoses_visit_id (visit_id=1)
        (cost=0.35 rows=1) (actual time=0.00799)
    -> Single-row index lookup on ds using PRIMARY (id=dg.disease_id)

```

MySQL wykorzystuje wyszukiwanie po indeksie i osiąga około 0.01 ms na poziomie silnika. Czas zaobserwowany w pomiarze właściwym (1.23 ms) jest zdominowany przez narzut protokołu klient-serwer i sterownika `mysql-connector-python`. To wyjaśnia minimalny wpływ indeksów w Tabeli 15 – InnoDB i tak buforuje całe tabele w puli buforów (4 GiB), więc różnica między sekwencyjnym przeglądem tabeli a wyszukiwaniem po indeksie ginie w szumie sieci i protokołu.

### 8.2.5 MongoDB – pobranie wizyt pacjenta z lekarzem (kolekcja wizyt 2M dok.)

Z indeksem na identyfikatorze lekarza:

```
winningPlan:
  stage: FETCH
  inputStage:
    stage: IXSCAN
    indexName: doctor_id_1
    keyPattern: { doctor_id: 1 }
totalDocsExamined: ~10; executionTimeMillis: ~1
```

Bez indeksu:

```
winningPlan:
  stage: COLLSCAN
  filter: { doctor_id: { $eq: 1 } }
totalDocsExamined: 2000000; executionTimeMillis: ~9
```

MongoDB przy 10M summarycznych rekordów (2 mln wizyt) wykonuje pełny przegląd dwumilionowej kolekcji bez indeksu – około 8.58 ms w pomiarze właściwym. Z indeksem silnik skanuje indeks i pobiera tylko pasujące dokumenty: 1.39 ms ( $S_{odczyt} = 6.18\times$ ).

### 8.2.6 Wnioski porównawcze

- **PostgreSQL** osiąga największe przyspieszenia przez indeksy:  $88\times$  w planie i pomiarze właściwym dla pobrania diagnoz wizyty przy 10M rekordów oraz  $22\text{--}71\times$  dla wyliczenia sumy kosztów usług. Optymalizator dobrze dobiera sposób dostępu do tabel w zależności od selektywności warunku.
- **MySQL** potrafi korzystać z indeksów, ale w tym pomiarze duże dodatkowe przyspieszenie nie jest widoczne w tabelach wyników, ponieważ kluczowe indeksy są już obecne z powodu ograniczeń integralności. Dlatego MySQL nie zachowuje się jak PostgreSQL bez indeksów; nawet wariant bazowy ma część struktur, które chronią go przed pełnym przeglądaniem tabel zależnych.
- **MongoDB** odnotowuje znaczący zysk tylko tam, gdzie zapytanie filtruje po polu innym niż identyfikator dokumentu, np. po identyfikatorze lekarza. Pozostałe scenariusze pozostają szybkie nawet bez dodatkowych indeksów, ponieważ model dokumentowy ogranicza potrzebę łączenia wielu kolekcji.

## 9 Weryfikacja hipotezy badawczej

**H1:** *Zastosowanie indeksów znacząco poprawia wydajność odczytu danych kosztem spadku wydajności wstawiania i aktualizacji danych. Oba efekty narastają ze skalą danych.*

Do kwantyfikacji efektu zastosowano dwie metryki:

- $S_{odczyt} = t_{bez}/t_z$  – współczynnik przyspieszenia odczytu ( $> 1$  = szybciej z indeksami),
- $K_{zapis} = t_z/t_{bez}$  – współczynnik spowolnienia zapisu ( $> 1$  = wolniej z indeksami).

## 9.1 Przyspieszenie odczytu ( $S_{odczyt}$ ) – trend ze skalą

Tabela 15: Współczynnik przyspieszenia odczytu dzięki indeksom ( $S_{odczyt}$ ). Redis pominięty – brak mechanizmu indeksów.

| Scenariusz                  | Baza       | 500k   | 1M     | 10M           |
|-----------------------------|------------|--------|--------|---------------|
| Wizyty pacjenta z lekarzem  | PostgreSQL | 6.75×  | 15.29× | <b>65.74×</b> |
| Wizyty pacjenta z lekarzem  | MySQL      | 0.92×  | 0.93×  | 0.98×         |
| Wizyty pacjenta z lekarzem  | MongoDB    | 1.20×  | 1.34×  | <b>5.95×</b>  |
| Diagnozy wizyty             | PostgreSQL | 6.63×  | 14.22× | <b>60.66×</b> |
| Pełna historia pacjenta     | PostgreSQL | 10.33× | 13.74× | <b>58.51×</b> |
| Suma kosztów usług pacjenta | PostgreSQL | 10.79× | 19.06× | <b>64.49×</b> |
| Recepty z lekami            | PostgreSQL | 6.72×  | 9.96×  | <b>72.02×</b> |

**Obserwacja:** przyspieszenie odczytu w PostgreSQL **narasta monotonicznie ze skalą** dla wszystkich scenariuszy z warunkiem filtrowania lub złączeniem po kolumnie klucza obcego – od około 7–11× przy 500k do **59–72×** przy 10M, z maksimum **72.02×** dla pobrania recept z lekami oraz **65.74×** dla pobrania wizyt pacjenta z lekarzem. MySQL pokazuje minimalny albo ujemny wpływ *dodatkowych* indeksów ( $S_{odczyt} \approx 0.9\text{--}1.0\times$ ), ponieważ InnoDB już w wariacie bazowym utrzymuje indeksy potrzebne do obsługi kluczy obcych, a badane zapytania w dużej części korzystają właśnie z tych powiązań. Wniosek nie brzmi więc, że indeksy w MySQL są zbędne, lecz że w tym schemacie najważniejszy efekt indeksowania był obecny przed dodaniem indeksów pomocniczych. MongoDB osiąga znaczący zysk głównie dla pobrania wizyt pacjenta z lekarzem przy 10M ( $S_{odczyt} = 5.95\times$ ); pozostałe odczyty dokumentowe są szybkie również bez dodatkowych indeksów, bo wynikają z modelu dokumentowego i dostępu po identyfikatorze.

## 9.2 Spowolnienie zapisu ( $K_{zapis}$ ) – trend ze skalą

Tabela 16: Współczynnik spowolnienia operacji zapisu z indeksami ( $K_{zapis}$ ).  $> 1$  = wolniej z indeksami.

| Scenariusz                    | Baza       | 500k  | 1M    | 10M          |
|-------------------------------|------------|-------|-------|--------------|
| Wstawienie pacjenta           | PostgreSQL | 1.05× | 0.93× | 0.90×        |
| Wstawienie wizyty             | PostgreSQL | 1.01× | 0.95× | 0.88×        |
| Wstawienie wizyty             | MySQL      | 1.25× | 1.26× | 0.99×        |
| Zmiana statusu wizyty         | PostgreSQL | 0.93× | 1.05× | <b>1.24×</b> |
| Zmiana ceny wykonanej usługi  | PostgreSQL | 0.11× | 0.09× | <b>0.01×</b> |
| Aktualizacja notatek diagnozy | PostgreSQL | 0.24× | 0.14× | <b>0.02×</b> |

**Obserwacja:** dwa różne reżimy zachowania aktualizacji i wstawiania danych z indeksami:

- **Klasyczne spowolnienie zapisu** – widoczne przy 10M dla zmiany statusu wizyty w PostgreSQL ( $K_{zapis} = 1.24$ , +24%) i MySQL ( $K_{zapis} = 1.67$ , +67%) – warunek dotyczy klucza głównego, więc dodatkowe indeksy stanowią czysty narzut przy zapisie. Pozostałe operacje wstawiania są neutralne lub lekko szybsze z indeksami ( $K_{zapis} = 0.88$ – $1.0$ ); brak klasycznego silnego spowolnienia wynika z ograniczonej liczby indeksów.
- **Przyspieszenie zapisu** ( $K_{zapis} \ll 1$ ) – widoczne dla zmiany ceny wykonanej usługi (PostgreSQL:  $K_{zapis} = 0.01\times$  przy 10M, czyli około  $72\times$  szybciej z indeksami) i aktualizacji notatek diagnozy ( $K_{zapis} = 0.02\times$ , około  $63\times$ ). Te scenariusze *nie kontrastują kosztu zapisu, lecz kosztu wyszukania* – warunek odwołuje się do kolumny klucza obcego, która bez indeksu wymaga sekwencyjnego przeglądu tabeli (67–97 ms przy 10M), a z indeksem wymaga wyszukania przez indeks i zapisu (0.9–1.5 ms).

## 9.3 Wnioski dotyczące hipotezy

Hipoteza H1 została **potwierdzona z istotnymi zastrzeżeniami wynikającymi z różnic między silnikami i modelem danych**. Najczytelniej potwierdza ją PostgreSQL: po dodaniu indeksów odczyty, aktualizacje i usunięcia wyszukujące rekordy po kolumnach powiązań przyspieszają wraz ze skalą nawet kilkadziesiąt albo kilkaset razy. MySQL nie pokazuje dużego dodatkowego zysku po dodaniu indeksów pomocniczych, ale wynika to z konstrukcji schematu i samego silnika: InnoDB już wcześniej utrzymuje indeksy wymagane przez klucze obce, więc wariant bazowy nie jest odpowiednikiem PostgreSQL pozbawionego indeksów na powiązaniach. MongoDB potwierdza hipotezę selektywnie – indeks pomaga wtedy, gdy zapytanie filtruje po konkretnym polu dokumentu, natomiast odczyty po identyfikatorze dokumentu pozostają szybkie bez dodatkowych struktur. Część dotycząca kosztu zapisu również wymaga doprecyzowania: dodatkowe indeksy spowalniają operacje, gdy rekord jest już znajdowany po kluczu głównym, ale mogą przyspieszyć aktualizację lub usunięcie, jeżeli największym kosztem jest wcześniejsze odnalezienie rekordów zależnych.



## 10 Podsumowanie i wnioski

1. **Redis** osiąga najniższą latencję ze wszystkich testowanych baz – operacje tworzenia, odczytu, aktualizacji i usuwania mieszczą się w przedziale **0.26–1.53 ms** dla wszystkich badanych skal. Wynika to z operacji w pamięci RAM i bezpośredniego dostępu po kluczu. Nie jest to jednak dowód przewagi w pełnym modelu transakcyjnym: Redis nie wykonuje złączeń, nie utrzymuje relacji i w tym pomiarze pracował bez trwałego zapisu na dysk.
2. **PostgreSQL jest najbardziej wrażliwy na poprawne indeksowanie.** Z indeksami osiąga bardzo dobre wyniki dla odczytu, aktualizacji po kolumnach powiązań i usuwania kaskadowego; bez indeksów te same operacje potrafią wzrosnąć do dziesiątek albo setek milisekund. To nie oznacza, że PostgreSQL jest ogólnie wolny, lecz że nie tworzy automatycznie wszystkich indeksów potrzebnych dla kluczy obcych.
3. **MongoDB ma stabilne wyniki tam, gdzie model dokumentowy pasuje do zapytania.** Odczyty i modyfikacje po identyfikatorze dokumentu pozostają szybko niezależnie od skali. Indeksy dają wyraźny zysk dopiero przy filtrowaniu po konkretnym polu zagnieżdżonym, np. przy pobraniu wizyt pacjenta z lekarzem na skali 10M ( $S_{odczyt} = 5.95\times$ ). Wniosek dla MongoDB nie brzmi więc „indeksy zawsze przyspieszają”, lecz „indeks musi odpowiadać rzeczywistemu polu filtrowania”.
4. **MySQL nie pokazał dużego dodatkowego przyspieszenia po dołożeniu indeksów pomocniczych.** Nie oznacza to jednak, że indeksy nie miały znaczenia: InnoDB tworzy indeksy wymagane przez klucze obce, więc już wariant bazowy korzysta z części najważniejszych struktur dostępu. Stabilne czasy MySQL należy więc interpretować jako efekt konstrukcji schematu i automatycznych indeksów, a nie jako zaprzeczenie roli indeksowania. Jednocześnie widać koszt zapisu przy aktualizacji po kluczu głównym: zmiana statusu wizyty przy 10M rekordów zwalnia o około 67%.
5. **Najważniejszy wniosek o indeksach potwierdza hipotezę, ale wymaga kontekstu.** Indeksy są krytyczne, gdy operacja musi znaleźć rekordy po kolumnach powiązań; wtedy eliminują pełne przeglądy tabel i dają największe zyski, zwłaszcza w PostgreSQL. W MySQL część tego efektu jest ukryta, bo indeksy kluczy obcych istnieją automatycznie, a w MongoDB efekt zależy od tego, czy zapytanie rzeczywiście filtruje po dodatkowym polu dokumentu. Nie są też automatycznie korzystne dla każdego zapisu: jeżeli rekord jest już znajdowany po kluczu głównym, dodatkowe indeksy mogą zwiększać koszt aktualizacji.
6. **Wybór SZBD** powinien być podyktowany charakterem aplikacji:
  - Złożone zapytania relacyjne i silna spójność transakcyjna → PostgreSQL z dobrze zaprojektowanymi indeksami,
  - Proste aplikacje internetowe i łatwość wdrożenia → MySQL,
  - Elastyczny schemat, dane hierarchiczne → MongoDB,
  - Najniższa latencja, pamięć podręczna, sesje, kolejki, dane tymczasowe → Redis.